



HotSchedules API Documentation

V9. Updated Mar 2025

Document History

Version	Date	Author	Change Description:
1.0	Mar 1, 2024	Chris Sundgren, Bohdan Kovalchuk, Volodymyr Illchenko	Aligned Format, updated error handling
2.0	Apr 16, 2024	Chris Sundgren	Added Doc History, updated getDriversByInterval to include SALES, ordered service methods to available services in table of contents
3.0	May 29, 2024	Chris Sundgren	Added setEmpV3 to document
4.0	October 1, 2024		Added getTimecardV5 to document
5.0	October 21, 2024	Illia Beskrovnyi	Added endpoints with alphanumeric store id: - getEmpAvailabilityV3 - setEmpsV4
6.0	October 24, 2024	Volodymyr Makarchuk	Added endpoints with alphanumeric store id: • setForecastDriversV2 • getEmpInfoV2 • getEmpJobsV2 • getScheduleV5
7.0	November 4, 2024	Ivan Synyshyn	Added getEmpAvailabilityV4 to document
8.0	January 31, 2025	Chris Sundgren	Added getLaborByPostionAndInterval
9.0	March 3, 2025	Volodymyr Makarchuk	Added getTimeCardsV6 with alphanumeric store id

Introduction

The Fourth SOAP API provides a user-friendly way to send all information about your store (employees, jobs, sales, etc.), or obtain shift and other valuable information from Fourth. Before you are able to make successful SOAP API calls, Fourth will need to provide the following to you:

1. Username and Password
2. Concept
3. Store ID

Methods allow you to send or retrieve data using a variety of requests (Fourth uses GET and SET requests, predominantly). **SET** requests will create or update existing data within Fourth, and **GET** requests will pull whatever data is in Fourth and present it to you.

Available Web Services (WSDLs)

Name	Description	Available Endpoints
EmpService	This service provides operations for a third party to push or request employee and employee job data into Fourth. The WSDL is available at: http://services.hotschedules.com/api/services/EmpService?wsdl	SayHello getCompanyEmployees getEmpAvailability getEmpAvailabilityV3 getEmpInfo getEmpInfoV2 getEmpJobs getEmpJobsV2 getEmpJobsV3 getEmpJobsV4 getSalariedJobs getStoreEmployees getStoreEmployeesV2 getStoreEmployeesV3 getStoreEmployeesV4 getStoreJobs getStoreJobsV2 setEmpJobs setEmpJobsV2 setEmpJobsV4 setEmps setEmpsV2 setEmpsV3 setEmpsV4 setSalariedJobs setAverageWage

Name	Description	Available Endpoints
ScheduleService	This service is intended for third parties to be able to grab scheduled shifts from Fourth and import them into their POS/data/warehouse/enterprise/etc. System. The WSDL is available at: http://services.hotschedules.com/api/services/ScheduleService?wsdl	sayHello getLocations getSchedule getSchedule2 getScheduleStatus getScheduleV3 getScheduleV4 getScheduleV5 getShiftsV3 getTemplates
TimeCardService	This service is intended for third parties to be able to import their time cards into the Fourth system or get time cards from Fourth in a straightforward fashion. The WSDL is available at: http://services.hotschedules.com/api/services/TimeCardService?wsdl	SayHello getTimeCards getTimeCardsDeclaredTips getTimeCardsV2 getTimeCardsV3 getTimeCardsV4Beta getTimeCardsV5 getTimeCardsV6 setTimeCards setTimeCardsDeclaredTips setTimeCardsV3 setTimeCardsv4 updateTimeCards
SalesService	This service is intended for third parties to be able to import their sales items into Fourth in a straightforward fashion. The WSDL is available at: http://services.hotschedules.com/api/services/SalesService?wsdl	sayHello getRVCs getSalesCats getSalesItemsV3 setSalesItems setSalesItemsV3 setSalesItemsV4
ProjectedSalesService	This service is intended for third parties to be able to request labor data from Fourth and import it into their POS/data/warehouse/enterprise/etc. system. The WSDL is available at: http://services.hotschedules.com/api/services/ProjectedSalesService?wsdl	sayHello getProjectedSales getProjectedSalesV3

	m /api/services/ProjectedSalesService?wsdl	
Name	Description	Available Endpoints
LaborService	This service is intended for third parties to be able to request labor data from Fourth and import it into their POS/data/warehouse/enterprise/etc. System. The WSDL is available at: http://services.hotschedules.com/api/services/LaborService?wsdl	sayHello getLaborByBusDate getLaborByJobAndInterval getLaborByJobAndIntervalV2 getLaborByPositionAndInterval
VolumeService	This service is intended for third parties to be able to request and send volume driver related data from/to Fourth. Driver items include examples such as guests, tables, entrees, etc. The WSDL is available at: http://services.hotschedules.com/api/services/VolumeService?wsdl	sayHello getDriversByInterval getGuestCounts getVolumeCounts setForecastDrivers setForecastDriversV2 setGuestCounts setVolumeCounts setVolumeCountsV2 setVolumeCountsV3 setVolumeCountsV4
TimeOffService	This service is intended for third parties to be able to request approved time off requests from Fourth. The WSDL is available at: http://services.hotschedules.com/api/services/TimeOffService?wsdl	sayHello getApprovedPTO
CertificationService	This service is intended for third parties to be able to request and set certification requests from Fourth. The WSDL is available at: http://services.hotschedules.com/api/services/CertificationService?wsdl	sayHello getClientCertifications getEmployeeCertifications removeEmployeeCertifications setEmployeeCertifications setEmployeeCertificationsV2
OperatingBudgetPlanService	The WSDL is available at: http://services.hotschedules.com	sayHello getLaborData

	/api/services/OperatingBudgetPlanService?wsdl	getLaborDataForWeek setLaborData
--	---	-------------------------------------

Name	Description	Available Endpoints
HierarchyService	The WSDL is available at: http://services.hotschedules.com/api/services/HierarchyService?wsdl	getConceptsV2 getGroups getStores

EmpService Available Methods

getEmpAvailability

This method takes in a concept ID, store ID and an active only flag then returns a list of all employee availability for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
activeOnly	Boolean	true	True = Return only active employees False = Return all employees

getem

Response:

Name	Type	Description/Example
dayName	String	Day of week.
dayNum	Int	Sunday = 1 Monday = 2 Tuesday = 3 Wednesday = 4 Thursday = 5 Friday = 6 Saturday = 7
parHoursMax	Int	Max Partial Hours
parHoursMin	Int	Minimum Partial Hours
parShiftsMax	Int	Max Partial Shifts
parShiftsMin	Int	Minimum Partial Shifts
shiftId	Int	Shift Internal ID
shiftName	String	Name of Shift, also known as day part.
statusName	String	Name of the employee's status: • Available • Not Available
statusNum	Int	Status ID: • 0 = Available • 3 = Not Available

Example getEmpAvailability Response:

```

<availabilities>
  <dayName>Sunday</dayName>
  <dayNum>1</dayNum>
  <parHoursMax>-1</parHoursMax>
  <parHoursMin>-1</parHoursMin>
  <parShiftsMax>-1</parShiftsMax>
  <parShiftsMin>-1</parShiftsMin>
  <partialBeforeAfter/>
  <partialTime/>
  <shiftId>781691596</shiftId>
  <shiftName>AM</shiftName>
  <statusName>Not Available</statusName>
  <statusNum>3</statusNum>
</availabilities>

```

getEmpAvailabilityV3

This method takes in a concept ID, store ID and an active only flag then returns a list of all employee availability for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeId	String	true	Alphanumeric Store ID
activeOnly	Boolean	true	True = Return only active employees False = Return all employees

Response:

Name	Type	Description/Example
dayName	String	Day of week.
dayNum	Int	Sunday = 1 Monday = 2 Tuesday = 3 Wednesday = 4 Thursday = 5 Friday = 6 Saturday = 7

parHoursMax	Int	Max Partial Hours
parHoursMin	Int	Minimum Partial Hours
parShiftsMax	Int	Max Partial Shifts
parShiftsMin	Int	Minimum Partial Shifts
shiftId	Int	Shift Internal ID
shiftName	String	Name of Shift, also known as day part.
statusName	String	Name of the employee's status: • Available • Not Available
statusNum	Int	Status ID: • 0 = Available • 3 = Not Available

Example getEmpAvailability Response:

```

<availabilities>
  <dayName>Sunday</dayName>
  <dayNum>1</dayNum>
  <parHoursMax>-1</parHoursMax>
  <parHoursMin>-1</parHoursMin>
  <parShiftsMax>-1</parShiftsMax>
  <parShiftsMin>-1</parShiftsMin>
  <partialBeforeAfter/>
  <partialTime/>
  <shiftId>781691596</shiftId>
  <shiftName>AM</shiftName>
  <statusName>Not Available</statusName>
  <statusNum>3</statusNum>
</availabilities>

```

getEmpAvailabilityV4

This method takes in a concept ID, store ID and an active only flag then returns a list of all employee availability for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.

storeId	String	true	Alphanumeric Store ID
activeOnly	Boolean	true	True = Return only active employees False = Return all employees

Response:

Name	Type	Description/Example
dayName	String	Day of week.
dayNum	Int	Sunday = 1 Monday = 2 Tuesday = 3 Wednesday = 4 Thursday = 5 Friday = 6 Saturday = 7
parHoursMax	Int	Max Partial Hours
parHoursMin	Int	Minimum Partial Hours
parShiftsMax	Int	Max Partial Shifts
parShiftsMin	Int	Minimum Partial Shifts
consecutiveDaysMax	Int	Maximum consecutive Days
daysInWeekMax	Int	Maximum days in week
daysInWeekMin	Int	Minimum days in week
hoursBetweenShiftMin	Int	Minimum hours between shifts
hoursInDayMax	Int	Maximum hours in day
hoursInWeekMax	Int	Maximum hours in week
hoursInWeekMin	Int	Minimum hours in week
shiftId	Int	Shift Internal ID
shiftName	String	Name of Shift, also known as day part.
statusName	String	Name of the employee's status: • Available • Not Available
statusNum	Int	Status ID: • 0 = Available • 3 = Not Available

Example getEmpAvailability Response:

```

<availabilities>
  <dayName>Sunday</dayName>
  <dayNum>1</dayNum>
  <parHoursMax>-1</parHoursMax>
  <parHoursMin>-1</parHoursMin>

```

```

<parShiftsMax>-1</parShiftsMax>
<parShiftsMin>-1</parShiftsMin>
<partialBeforeAfter/>
<partialTime/>
<shiftId>781691596</shiftId>
<shiftName>AM</shiftName>
<statusName>Not Available</statusName>
<statusNum>3</statusNum>
</availabilities>
  <threshold>
    <consecutiveDaysMax>3</consecutiveDaysMax>
    <daysInWeekMax>2</daysInWeekMax>
    <daysInWeekMin>1</daysInWeekMin>
    <hoursBetweenShiftMin>1.0</hoursBetweenShiftMin>
    <hoursInDayMax>1.0</hoursInDayMax>
    <hoursInWeekMax>2.0</hoursInWeekMax>
    <hoursInWeekMin>1.0</hoursInWeekMin>
  </threshold>

```

getEmpInfo

This method takes in a concept ID, a store ID and an active only flag, then returns a list of all employees for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
activeOnly	Boolean	true	True or false . Return only active employees or not.

Response:

Name	Type	Description/Example
accountCreated	String	Date and time employee was created
assignedSchedules	Array	- extId - External employee ID - hsId - Internal employee ID - name - Schedule name
empHrId	Int	Employee HR ID

empNum	Int	Employee POS ID
lastUpdated	String	Last time employee was updated
permissionSetName	String	The permission set assigned to employee

Example getEmpInfo Response:

```

<item>
  <accountCreated>2019-01-14T14:56:05.537-06:00</accountCreated>
  <empHrId>-1</empHrId>
  <empNum>-1</empNum>
  <permissionSetName>Manager</permissionSetName>
</item>
<item>
  <accountCreated>2012-03-09T15:51:51.650-06:00</accountCreated>
  <empHrId>-1</empHrId>
  <empNum>-1</empNum>
  <lastUpdated>2018-01-31T22:09:32.923-06:00</lastUpdated>
  <permissionSetName>Default HS Support User</permissionSetName>
</item>
<item>
  <accountCreated>2012-03-23T13:12:49.457-05:00</accountCreated>
  <assignedSchedules>
    <extId>0</extId>
    <hsId>781691494</hsId>
    <name>Busser</name>
  </assignedSchedules>
  <assignedSchedules>
    <extId>0</extId>
    <hsId>781691498</hsId>
    <name>Host</name>
  </assignedSchedules>
  <assignedSchedules>
    <extId>0</extId>
    <hsId>781691539</hsId>
    <name>Kitchen</name>
  </assignedSchedules>

```

```

    <empHrId>-1</empHrId>
    <empNum>1013</empNum>
    <lastUpdated>2017-05-18T09:33:15.057-05:00</lastUpdated>
    <permissionSetName>Employee</permissionSetName>
  </item>

```

getEmpInfoV2

This method takes in a concept ID, an alphanumeric store ID and an active only flag, then returns a list of all employees for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeId	String	true	Alphanumeric Store ID
activeOnly	Boolean	true	True or false . Return only active employees or not.

Response:

Name	Type	Description/Example
accountCreated	String	Date and time employee was created
assignedSchedules	Array	- extId - External employee ID - hsId - Internal employee ID - name - Schedule name
empHrId	Int	Employee HR ID
empNum	Int	Employee POS ID
lastUpdated	String	Last time employee was updated
permissionSetName	String	The permission set assigned to employee

Example getEmpInfoV2 Response:

```

<item>
  <accountCreated>2019-01-14T14:56:05.537-06:00</accountCreated>
  <empHrId>-1</empHrId>
  <empNum>-1</empNum>
  <permissionSetName>Manager</permissionSetName>
</item>
<item>

```

```

    <accountCreated>2012-03-09T15:51:51.650-06:00</accountCreated>
    <empHrId>-1</empHrId>
    <empNum>-1</empNum>
    <lastUpdated>2018-01-31T22:09:32.923-06:00</lastUpdated>
    <permissionSetName>Default HS Support User</permissionSetName>
</item>
<item>
    <accountCreated>2012-03-23T13:12:49.457-05:00</accountCreated>
    <assignedSchedules>
        <extId>0</extId>
        <hsId>781691494</hsId>
        <name>Busser</name>
    </assignedSchedules>
    <assignedSchedules>
        <extId>0</extId>
        <hsId>781691498</hsId>
        <name>Host</name>
    </assignedSchedules>
    <assignedSchedules>
        <extId>0</extId>
        <hsId>781691539</hsId>
        <name>Kitchen</name>
    </assignedSchedules>
    <empHrId>-1</empHrId>
    <empNum>1013</empNum>
    <lastUpdated>2017-05-18T09:33:15.057-05:00</lastUpdated>
    <permissionSetName>Employee</permissionSetName>
</item>

```

getEmpJobs

This method takes in a concept ID and a store ID and returns a list of all jobs assigned to all employees for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID

Response:

Name	Type	Description/Example
clientId	Int	Unique Identifier for client provided by Fourth
hsEmpId	Int	Fourth internal employee account ID
hsJobId	Int	Fourth internal job code ID
ovtWage	Float	Overtime hourly wage rate for employee
posEmpId	Int	POS ID for employee
posJobId	Int	POS ID for the job code
primary	Boolean	true = If employee's primary job false = if not employee's primary job
regWage	Float	Regular hourly wage rate for employee
storeNum	Int	Store ID

Example getEmpJobs Response:

```

<item>
  <clientId>5967459</clientId>
  <hsEmpId>3010240</hsEmpId>
  <hsJobId>16921407</hsJobId>
  <ovtWage>13.875</ovtWage>
  <posEmpId>1013</posEmpId>
  <posJobId>-1</posJobId>
  <primary>>false</primary>
  <regWage>9.25</regWage>
  <storeNum>2</storeNum>
</item>

```

getEmpJobsV2

This method takes in a concept ID and a alphanumeric store ID and returns a list of all jobs assigned to all employees for that store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeId	String	true	Alphanumeric Store ID

Response:

Name	Type	Description/Example
clientId	Int	Unique Identifier for client provided by Fourth
hsEmpId	Int	Fourth internal employee account ID
hsJobId	Int	Fourth internal job code ID
ovtWage	Float	Overtime hourly wage rate for employee
posEmpId	Int	POS ID for employee
posJobId	Int	POS ID for the job code
primary	Boolean	true = If employee's primary job false = if not employee's primary job
regWage	Float	Regular hourly wage rate for employee
storeNum	Int	Store ID

Example getEmpJobsV2 Response:

```

<item>
  <clientId>5967459</clientId>
  <hsEmpId>3010240</hsEmpId>
  <hsJobId>16921407</hsJobId>
  <ovtWage>13.875</ovtWage>
  <posEmpId>1013</posEmpId>
  <posJobId>-1</posJobId>
  <primary>false</primary>
  <regWage>9.25</regWage>
  <storeNum>2</storeNum>
</item>

```

getCompanyEmployees

This service is intended for third parties to be able to get all Company Employees filtering active and above store console employees. This method takes in two boolean values: activeOnly and aboveStore.

Input (Literal)

The inputs of this method are the arguments defined by the following table.

Content Model

Argument	Type	Occurs	Description
All		1..1	
activeOnly	boolean	1..1	If true – only active employees are returned. If false – all statuses are shown (active/inactive/terminated).
aboveStore	boolean	1..1	If true –ASC users are also returned with other users. If false – no ASC users are returned.

Output (Literal)

The outputs of this method are the arguments defined by the following table.

Argument	Type	Occurs	Description
All	wsReturn	1..1	WSReturn object

Example

SAMPLE CALL:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:emp="http://services.hotschedules.com/api/services/EmpService">
  <soapenv:Header/>
  <soapenv:Body>
    <emp:getCompanyEmployees>
      <activeOnly>false</activeOnly>
      <aboveStore>true</aboveStore>
    </emp:getCompanyEmployees>
  </soapenv:Body>
</soapenv:Envelope>
```

RESPONSE:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:getCompanyEmployeesResponse
      xmlns:ns1="http://services.hotschedules.com/api/services/EmpService">
      <return>
        <item>
          <aboveStore>false</aboveStore>
          <alphanumericStoreId>test alpha 24312431</alphanumericStoreId>
          <firstName>Ivan2</firstName>
          <homeStore>true</homeStore>
          <hrId/>
          <hsId>1775470613</hsId>
          <lastName>1211</lastName>
          <posId>714334883</posId>
          <status>-1</status>
          <storeId>65322</storeId>
        </item>
        <item>
          <aboveStore>false</aboveStore>
          <alphanumericStoreId>test 231</alphanumericStoreId>
          <firstName>Ivan2122</firstName>
          <homeStore>true</homeStore>
          <hrId/>
          <hsId>1775613</hsId>
          <lastName>Test In the CLub</lastName>
          <posId>714383</posId>
          <status>1</status>
          <storeId>65322</storeId>
        </item>
        <item>
          <aboveStore>true</aboveStore>
          <alphanumericStoreId>0</alphanumericStoreId>
          <firstName>test</firstName>
          <homeStore>false</homeStore>
          <hrId/>
          <hsId>1868918192</hsId>
          <lastName>asc</lastName>
          <posId>-1</posId>
          <status>0</status>
          <storeId>0</storeId>
        </item>
      </return>
    </ns1:getCompanyEmployeesResponse>
  </soap:Body>
</soap:Envelope>
```

getSalariedJobs

This method takes in a concept and store ID, then provides a list of all of the salaried jobs within the specified location.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID

Response:

Name	Type	Description/Example
annualSalary	Int	Annual salary of job.
clientID	Int	Unique Identifier for client provided by Fourth
hsEmpId	Int	Fourth internal employee account ID
hsJobId	Int	Fourth internal job code ID
posEmpId	Int	Employee POS ID
posJobId	Int	POS ID for the job code
storeNum	Int	Store ID

Example getSalariedJobs Response:

```
<item>
  <annualSalary>5000000</annualSalary>
  <clientId>5967459</clientId>
  <hsEmpId>3028758</hsEmpId>
  <hsJobId>8965152</hsJobId>
  <posEmpId>1070</posEmpId>
  <posJobId>11</posJobId>
  <storeNum>2</storeNum>
</item>
```

getStoreEmployees

This method takes in a concept ID, store ID and a flag to determine if only active employees are returned.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
activeOnly	Boolean	true	True or false . Return only active employees or not.

Response:

Name	Type	Description/Example
address	String	Employee address
address2	String	Employee Address Line 2
altId	Int	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.
city	String	City Field for Address
clientId	Int	Unique Identifier for client provided by Fourth
email	String	Employee's Email Address
empNum	Int	Employee POS ID
FName	String	Employee First Name
hsId	Int	Fourth internal employee account ID
LName	String	Employee Last Name
mobile	String	Employees Mobile Phone
NName	String	Employee Nickname

phone	String	Phone Number
state	String	State
status	Int	Active = 1, Inactive = 0, Terminated = -1
storeNum	Int	Store ID
zipCode	Int	Zip Code
dob	String	Date of Birth
hireDate	String	Employee Hire Date

Example getStoreEmployees Response:

```

<item>
  <address>1042 HOLLYWOOD AVE</address>
  <address2/>
  <altId>-1</altId>
  <city>Austin</city>
  <clientId>5967459</clientId>
  <email>grillme@mailtothis.com</email>
  <empNum>-1</empNum>
<FName>Fourth</FName>
  <hsId>2973578</hsId>
  <LName>Support</LName>
  <mobile/>
  <NName/>
  <phone>(512) 219-1492</phone>
  <state>TX</state>
  <status>1</status>
  <storeNum>2</storeNum>
  <zipCode>78730</zipCode>
  <dob>2000-02-01T00:00:00-06:00</dob>
  <hireDate>2012-03-09T00:00:00-06:00</hireDate>
</item>

```

getStoreEmployeesV2

This method takes in a concept ID, store ID and a flag to determine if only active employees are returned.

Call:

Name	Type	Required	Description/Example
Concept	Int	true	Concept ID - Provided by Fourth.
StoreNum	Int	true	Store ID
activeOnly	Boolean	true	True = returns only active employees False = returns all employees

Response:

Name	Type	Description/Example
address	String	Employee address
address2	String	Employee Address Line 2
altId	Int	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.
city	String	City Field for Address
clientId	Int	Unique Identifier for client provided by Fourth
email	String	Employee's Email Address
empNum	Int	Employee POS ID
FName	String	Employee First Name
hsId	Int	Fourth internal employee account ID
LName	String	Employee Last Name
mobile	String	Employees Mobile Phone
NName	String	Employee Nickname
phone	String	Phone Number
state	String	State
status	Int	Active = 1, Inactive = 0, Terminated = -1
storeNum	Int	Store ID
zipCode	Int	Zip Code
dob	String	Date of Birth
hireDate	String	Employee Hire Date
payType	String	Hourly or Salary

Example get Response:

```
<item>
  <address/>
  <address2/>
  <altId>-1</altId>
  <city/>
  <clientId>5967459</clientId>
```



```

<email>michelle.gross@Fourth.com</email>
<empNum>-1</empNum>
<FName>Michelle</FName>
<hsId>741457</hsId>
<LName>Gross</LName>
<mobile/>
<NName/>
<phone>no number</phone>
<state/>
<status>1</status>
<storeNum>2</storeNum>
<zipCode/>
<dob>1969-12-31T00:00:00-06:00</dob>
<hireDate>2019-01-14T14:56:05.537-06:00</hireDate>
<customFields/>
<payType>hourly</payType>
</item>

```

getStoreJobs

This method takes in a concept ID and a store ID, and returns an array of all jobs currently defined in Fourth for the given concept/store.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID

Response:

Name	Type	Description/Example
clientId	Int	Unique Identifier for client provided by Fourth
defRate	Float	Default Pay Rate
hsId	Int	Fourth internal job ID
jobName	String	Name of job code
posId	Int	POS ID for the job code
storeNum	Int	Store ID

Example getStoreJobs Response:

```
<item>
  <clientId>5967459</clientId>
  <defRate>8.7</defRate>
  <hsId>786323127</hsId>
  <jobName>Bartender</jobName>
  <posId>18</posId>
  <storeNum>2</storeNum>
</item>
```

setEmpJobs

This method takes in a concept ID, store ID, and an array of objects to assign jobs to individual employees.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
clientId	Int	false	Unique Identifier for client provided by Fourth
hsEmpId	Int	false	Fourth internal employee account ID
hsJobId	Int	false	Fourth internal job code ID
ovtWage	Float	true	Overtime hourly wage rate for employee
posEmpId	Int	true	POS ID for employee
posJobId	Int	true	POS ID for the job code
primary	Boolean	true	true = If employee's primary job false = if not employee's primary job
regWage	Float	true	Regular hourly wage rate for employee
storeNum	Int	true	Store ID
date	String	false	Today's date (ex: 2019-11-22T00:00:00Z)

Example setEmpJobs Call:

```
<emp:setEmpJobs>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
```

```

<jobs>
  <item>
    <clientId>20390791</clientId>
    <hsEmpId>24930346</hsEmpId>
    <ovtWage>13.50</ovtWage>
    <posEmpId>85</posEmpId>
    <posJobId>1</posJobId>
    <primary>true</primary>
    <regWage>9.00</regWage>
    <storeNum>1100030</storeNum>
    <date>2019-11-22T00:00:00Z</date>
  </item>
</jobs>
</emp:setEmpJobs>

```

setEmpJobsV2

This method takes in a concept ID, store ID, and an array of objects to assign jobs to individual employees.

get

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
clientId	Int	false	Unique Identifier for client provided by Fourth
hsEmpId	Int	false	Fourth internal employee account ID
hsJobId	Int	false	Fourth internal job code ID
ovtWage	Float	false	Overtime hourly wage rate for employee
posEmpId	Int	true	POS ID for employee
posJobId	Int	true	POS ID for the job code
primary	Boolean	true	true = If employee's primary job false = if not employee's primary job
regWage	Float	true	Regular hourly wage rate for employee
storeNum	Int	false	Store ID
date	String	false	Today's date (ex: 2019-11-22T00:00:00Z)

skillLevel	Int	false	Skill Rank. For lowest rank use 0.
------------	-----	-------	------------------------------------

Example setEmpJobsV2 Call:

```

<emp:setEmpJobs>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
  <jobs>
    <item>
      <clientId>20390791</clientId>
      <hsEmpId>24930346</hsEmpId>
      <ovtWage>13.50</ovtWage>
      <posEmpId>85</posEmpId>
      <posJobId>1</posJobId>
      <primary>true</primary>
      <regWage>9.00</regWage>
      <storeNum>1100030</storeNum>
      <date>2019-11-22T00:00:00Z</date>
    </item>
  </jobs>
</emp:setEmpJobs>

```

setEmpJobsV4

This method takes in a concept ID, store ID, and an array of objects to assign jobs to individual employees.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeId	String	true	Alphanumeric Store ID
clientId	Int	false	Unique Identifier for client provided by Fourth
hsEmpId	Int	false	Fourth internal employee account ID
hsJobId	Int	false	Fourth internal job code ID

ovtWage	Float	true	Overtime hourly wage rate for employee
posEmpId	Int	true	POS ID for employee
posJobId	Int	true	POS ID for the job code
primary	Boolean	true	true = If employee's primary job false = if not employee's primary job
regWage	Float	true	Regular hourly wage rate for employee
storeNum	Int	false	Store ID
date	String	false	Today's date (ex: 2019-11-22T00:00:00Z)

Example setEmpJobsV4 Call:

```

<emp:setEmpJobs>
  <concept>280</concept>
  <storeId>store123456</storeId>
  <jobs>
    <item>
      <clientId>20390791</clientId>
      <hsEmpId>24930346</hsEmpId>
      <ovtWage>13.50</ovtWage>
      <posEmpId>85</posEmpId>
      <posJobId>1</posJobId>
      <primary>true</primary>
      <regWage>9.00</regWage>
      <date>2019-11-22T00:00:00Z</date>
    </item>
  </jobs>
</emp:setEmpJobs>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client does not have permission to use the EmpService operation: {}
    </faultstring>
  </soap:Body>
</soap:Envelope>

```

```

    <detail>
      <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/EmpService"/>
    </detail>
  </soap:Fault>
</soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element:

faultstring	Description
client does not have permission to use the EmpService Set Employees operation { }	Insufficient permissions to use the API. Please contact Fourth to allow access.
client not found for concept: { } store: { }	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
jobs == null	Jobs are missing. Please ensure that the params are correct and resubmit the request.
fatal error in employee job list insert/update: { }	Please ensure that the params are correct and resubmit the request. Contact Fourth support if problem persist
No skill levels defined	Skill levels not configured for client. Please ensure that the params are correct and resubmit the request. Contact Fourth support if problem persist

Partially successful responses

The API will attempt to process all the employeeJobs information that pass validation. Invalid job items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of <jobs> items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmpJobsV2Response xmlns:ns1="http://services.hotschedules.com/api/services/EmpService ">
      <return>
        <errors> EmpJob for employee [posEmpId] will not be synced, clientId may not be <
0</errors>

```

```

    <failCount>1</failCount>
    <success>>false</success>
    <successCount>0</successCount>
  </return>
</ns1: setEmpJobsV2Response>
</soap:Body>
</soap:Envelope>

```

Validation error	Description
EmpJob for employee [posEmpId] will not be synced, clientId may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, hsEmpId may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, hsJobId may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, posEmpId may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, posJobId may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, regWage may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, ovtWage may not be < 0	Please ensure that the params are correct and resubmit the request.
EmpJob for employee [posEmpId] will not be synced, skillLevel must be between 1 and 5	Please ensure that the params are correct and resubmit the request.

setEmps

This method takes in a concept ID, store ID, and an array of objects to provide the basic information for each employee at the specified location to Fourth.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
emps	Array	true	Each object is an employee at this store. See below for details about the array.

Employee Array

Name	Type	Required	Description/Example
zipCode	Int	false	Zip Code
hireDate	String	false	Employee Hire Date (ex: 2015-09-16T00:00:00-05:00)
termDate	String	false	Employee Termination Date (ex: 2019-09-16T00:00:00-05:00)
address	String	false	Employee Address Line 1
clientId	Int	true	Unique Identifier for client provided by Fourth
LName	String	true	Employee Last Name
address2	String	false	Employee Address Line 2
city	String	false	City Field for Address
mobile	String	false	Employees Mobile Phone
NName	String	false	Employee Nickname
altId	Int	false	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.
FName	String	true	Employee First Name
empNum	Int	true	Employee POS ID
phone	String	false	Phone Number
hsId	Int	false	Fourth internal employee account ID
dob	String	false	Date of Birth
state	String	false	State
storeNum	Int	true	Store ID
email	String	false	Employee's Email Address
status	Int	true	- Active = 1 - Inactive = 0 - Terminated = -1
alphanumericHrId	String	false	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.

Example setEmps call:

```

<emp:setEmps>
  <concept>1234</concept>
  <storeNum>10</storeNum>
  <emps>
    <item>
      <address>1234 Elm Street</address>
      <city>Springfield</city>
    
```



```

        <empNum>50</empNum>
<FName>James</FName>
        <LName>Seagall</LName>
        <mobile>222-222-2222</mobile>
        <NName>Jim</NName>
        <state>MO</state>
        <status>1</status>
        <storeNum>10</storeNum>
        <hireDate>2015-09-16T00:00:00-05:00</hireDate>
    </item>
</emps>
</emp:setEmps>

```

setEmpsV2

This method takes in a concept ID, store ID, and an array of objects to provide the basic information for each employee at the specified location to Fourth.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
emps	Array	true	Each object is an employee at this store. See below for details about the array.

setEmpsV2 Array

Name	Type	Required	Description/Example
address	String	false	Employee Address Line 1
address2	String	false	Employee Address Line 2
altId	Int	false	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.
city	String	false	City Field for Address
clientId	Int	false	Unique Identifier for client provided by Fourth
email	String	false	Employee's Email Address
empNum	Int	true	Employee POS ID
FName	String	false	Employee First name
hsId	Int	false	Fourth internal employee account ID

LName	String	false	Employee Last Name
mobile	String	false	Employees Mobile Phone
NName	String	false	Employee Nickname
phone	String	false	Phone Number
state	String	false	State
status	Int	true	Active = 1, Inactive = 0, Terminated = -1
storeNum	Int	false	Store ID
zipCode	Int	false	Zip Code
alphanumericId	String	false	Employee HR ID. This must be a unique value across the company. It is used for Fourth Shared Employees.
dob	String	false	Employees Date of Birth
hireDate	String	false	Employee Hire Date (ex: 2015-09-16T00:00:00-05:00)
termDate	String	false	Employee Termination Date (ex: 2019-09-16T00:00:00-05:00)
customField	String	false	This field needs to exist in the above store console first before a value can be sent to it. • Name = Custom Field's Name • Value = Custom Field's Value *See example below.
HomeStore	Boolean	false	Is this store the employee's home store? (true or false)
payType	string	false	hourly or salaried

Example setEmpsV2 call:

```

<emps>
  <item>
    <address>123 Elmwood Ln</address>
    <address2>Apt 32</address2>
    <city>Austin</city>
    <email>birch564@gmail.com</email>
    <empNum>55555</empNum>
    <FName>Edgar</FName>
    <LName>Allen</LName>
    <mobile>555-555-5556</mobile>
    <NName>Ed</NName>
  
```

```

    <phone>555-555-5555</phone>
    <state>TX</state>
    <status>1</status>
    <zipCode>78701</zipCode>
    <alphanumericHrId>EGR1324</alphanumericHrId>
    <dob>1987-01-01</dob>
    <hireDate>2020-01-01</hireDate>
    <customFields>
      <customField name="Birch ID" value="32"/>
    </customFields>
    <homeStore>true</homeStore>
    <payType>hourly</payType>
  </item>
</emps>

```

setEmpsV3

This method takes in a concept ID, store ID, and an array of objects to provide the basic information for each employee at the specified location to Fourth.

Call:

Name	Type	Occurs	Description
All		1..1	
concept	int	1..1	The identifier for the location's concept/group. Must be unique within the company. Contact v3if you're not sure about this value.
storeNum	int	1..1	Numeric (integer) identifier for the location. Must be unique within the concept.

emps	wsEmployee5Array	1..1	Array of WSEmployee5 objects. Each object represents an employee at this store.
------	------------------	------	---

SetEmpsV3 Array

Name	Type	Occurs	Required	Description
SEQUENCE				
address	string	0..1	No	Address Line 1
address2	string	0..1	No	Address Line 2
altId	int	1..1	No	Alternative ID for employee generally reserved for the HR ID in case company doesn't use alphanumeric HR ID
city	string	0..1	No	City field for Address
clientId	int	1..1	No	Unique client ID provided via HotSchedules
email	string	0..1	No	Employee email address
empNum	int	1..1	No	Employee POS ID
FName	string	0..1	No	Employee first name

hsId	int	1..1	No	HotSchedules Internal employee account ID -1 for newly created Employees
LName	string	0..1	No	Employee last name
mobile	string	0..1	No	Employee mobile phone number
NName	string	0..1	No	Employee nick name (preferred name)
phone	string	0..1	No	Employee phone number
state	string	0..1	No	State field for Address
status	int	1..1	No	Employee status Active = 1, Inactive = 0, Terminated = -1
storeNum	int	1..1	No	Numeric (integer) identifier for the location. Must be unique within the concept.
zipCode	string	0..1	No	Zip code field for Address
alphanumericHrId	string	0..1	No	Alphanumeric HR ID for employee
dob	dateTime	0..1	No	Employee Date of Birth
hireDate	dateTime	0..1	No	Employee Hire Date
termDate	dateTime	0..1	No	Employee termination date

customFields	customFieldArray	1..1	No	Accepts name and value parameters for custom fields
homeStore	boolean	1..1	No	Flag to indicate Employee Home Store
payType	string	1..1	No	Employee Pay Type: hourly or salaried
otExempt	boolean	1..1	No	Flag to indicate if salaried employee is overtime exempt or not exempt
salaryAmount	int	1..1	No	Employee salary as integer value with cents For salary 150 000 \$ put 15000000
salaryFrequency	int	1..1	No	Salary frequency hourly = 0 weekly = 1 bi-weekly = 2 monthly = 3 annual = 4

SAMPLE CALL:

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:emp="http://services.hotschedules.com/api/services/EmpService"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header>
<wsse:Security
xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0. xsd"
xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xs d"
soapenv:mustUnderstand="1">
  < wsse:UsernameToken wsu:Id="UsernameToken-63A716DE011BE2D649145262747932814">
    <wsse:Username>laura1234!</wsse:Username>
    <wsse:Password
Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#
PasswordText">laura 1234!</wsse:Password>
    <wsse:Nonce
EncodingType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-soap-message-sec
urity-1.0#Base64Binary">9iOMGex3ZDEpdw4xsAweWA==</wsse:Nonce>
    <wsu:Created>2016-01-12T19:37:59.328Z</wsu:Created>
```

```

    < /wsse:UsernameToken>
  </wsse:Security>
<soapenv:Header/>
<soapenv:Body>
  <emp:setEmpsV3>
    <concept>111</concept>
    <storeNum>222</storeNum>
    <emps>
      <!--Zero or more repetitions:-->
      <item>
        <!--Optional:-->
        <address>Test Address</address>
        <!--Optional:-->
        <address2>Test Address 2</address2>
        <!--Optional:-->
        <altId>10</altId>
        <!--Optional:-->
        <city>Austin</city>
        <clientId>123456789</clientId>
        <!--Optional:-->
        <email>test@email.com</email>
        <empNum>100</empNum>
        <!--Optional:-->
        <FName>Sandra</FName>
        <hsId>-1</hsId>
        <!--Optional:-->
        <LName>White</LName>
        <!--Optional:-->
        <mobile>(405) 567-2567</mobile>
        <!--Optional:-->
        <NName>pref name Sandra</NName>
        <!--Optional:-->
        <phone>(843) 883-4228</phone>
        <!--Optional:-->
        <state>TEXAS</state>
        <status>1</status>
        <storeNum>222</storeNum>
        <!--Optional:-->
        <zipCode>75104</zipCode>
        <!--Optional:-->
        <alphanumericHrId>10alpha</alphanumericHrId>
        <!--Optional:-->
        <dob>1988-04-01T00:00:00</dob>
        <!--Optional:-->
        <hireDate>2021-02-04T00:00:00</hireDate>
      </item>
    </emps>
  </emp:setEmpsV3>
</soapenv:Body>
</soapenv:Header>
</wsse:Security>
</wsse:UsernameToken>

```

```

<!--Optional:-->
<termDate>2024-02-04T00:00:00</termDate>
<!--Optional:-->
<customFields>
  <!--Zero or more repetitions:-->
  <customField name="customField" value="anyValue"/>
</customFields>
<!--Optional:-->
<homeStore>true</homeStore>
<!--Optional:-->
<payType>salaried</payType>
<!--Optional:-->
<otExempt>true</otExempt>
<!--Optional:-->
<salaryAmount>15000000</salaryAmount>
<!--Optional:-->
<salaryFrequency>4</salaryFrequency>
</item>
</emps>
</emp:setEmpsV3>
</soapenv:Body>
</soapenv:Envelope>

```

RESPONSE:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmpsV3Response
xmlns:ns1="http://services.hotschedules.com/api/services/EmpService">
      <return>
        <failCount>0</failCount>
        <success>true</success>
        <successCount>1</successCount>
      </return>
    </ns1:setEmpsV3Response>
  </soap:Body>
</soap:Envelope>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>

```



```

<soap:Fault>
  <faultcode>soap:Server</faultcode>
  <faultstring> client does not have permission to use the EmpService operation: {}
</faultstring>
  <detail>
    <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/EmpService"/>
  </detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element:

faultstring	Description
client does not have permission to use the EmpService Set Employees operation {}	Insufficient permissions to use the API. Please contact Fourth to allow access.
client not found for concept: {} store: {}	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
Edit Employee Pay Type permission is not enabled for client	Insufficient permissions to use the API. Please contact Fourth to allow access.
Found invalid Pay Type value: {}. Only 'hourly' and 'salaried' Pay Types are allowed.	Invalid Pay Type. Please ensure that the params are correct and resubmit the request.
Employee Management is not enabled for client	Insufficient permissions to use the API. Please contact Fourth to allow access.
Cannot identify employee {} /Employee {} is not found	Employee not found, please check that employee exists and correctly mapped by pos id or hrid. Please ensure that the params are correct and resubmit the request.
{} is not a valid Employee Type.	Wrong employee type in request, check payType in request (hourly/salaried). Please ensure that the params are correct and resubmit the request.

Partially successful responses

The API will attempt to process all the employee information that pass validation. Invalid employee items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of <emps> items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmpsV3Response xmlns:ns1="http://services.hotschedules.com/api/services/
EmpService ">
      <return>
        <errors> error parsing employee: {}</errors>
        <failCount>1</failCount>
        <success>false</success>
        <successCount>0</successCount>
      </return>
    </ns1: setEmpsV3Response>
  </soap:Body>
</soap:Envelope>
```

Validation error	Description
error parsing employee: {}	Please ensure that the params are correct and resubmit the request. Contact Fourth support
fatal error in employee list insert/update: {}	Please ensure that the params are correct and resubmit the request. Contact Fourth support

setEmpsV4

This method takes in a concept ID, store ID, and an array of objects to provide the basic information for each employee at the specified location to Fourth.

Call:

Name	Type	Occurs	Description
All		1..1	

concept	int	1..1	The identifier for the location's concept/group. Must be unique within the company. Contact v3if you're not sure about this value.
storeId	String	1..1	Alphanumeric Store ID
emps	wsEmployee5Array	1..1	Array of WSEmployee5 objects. Each object represents an employee at this store.

SetEmpsV4 Array

Name	Type	Occurs	Required	Description
SEQUENCE				
address	string	0..1	No	Address Line 1
address2	string	0..1	No	Address Line 2
altId	int	1..1	No	Alternative ID for employee generally reserved for the HR ID in case company doesn't use alphanumeric HR ID
city	string	0..1	No	City field for Address
clientId	int	1..1	No	Unique client ID provided via HotSchedules

email	string	0..1	No	Employee email address
empNum	int	1..1	No	Employee POS ID
FName	string	0..1	No	Employee first name
hsId	int	1..1	No	HotSchedules Internal employee account ID -1 for newly created Employees
LName	string	0..1	No	Employee last name
mobile	string	0..1	No	Employee mobile phone number
NName	string	0..1	No	Employee nick name (preferred name)
phone	string	0..1	No	Employee phone number
state	string	0..1	No	State field for Address
status	int	1..1	No	Employee status Active = 1, Inactive = 0, Terminated = -1
storeNum	int	1..1	No	Numeric (integer) identifier for the location. Not used in this endpoint
zipCode	string	0..1	No	Zip code field for Address
alphanumericHrId	string	0..1	No	Alphanumeric HR ID for employee
dob	dateTime	0..1	No	Employee Date of Birth

hireDate	dateTime	0..1	No	Employee Hire Date
termDate	dateTime	0..1	No	Employee termination date
customFields	customFieldArray	1..1	No	Accepts name and value parameters for custom fields
homeStore	boolean	1..1	No	Flag to indicate Employee Home Store
payType	string	1..1	No	Employee Pay Type: hourly or salaried
otExempt	boolean	1..1	No	Flag to indicate if salaried employee is overtime exempt or not exempt
salaryAmount	int	1..1	No	Employee salary as integer value with cents For salary 150 000 \$ put 15000000
salaryFrequency	int	1..1	No	Salary frequency hourly = 0 weekly = 1 bi-weekly = 2 monthly = 3 annual = 4

SAMPLE CALL:

```

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:emp="http://services.hotschedules.com/api/services/EmpService"
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header>
    <wsse:Security
      xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd"
      xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
      soapenv:mustUnderstand="1">
        <wsse:UsernameToken wsu:Id="UsernameToken-63A716DE011BE2D649145262747932814">
          <wsse:Username>laura1234!</wsse:Username>
          <wsse:Password

```

```

Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0#
PasswordText">laura 1234!</wsse:Password>
    <wsse:Nonce
EncodingType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-soap-message-sec
urity-1.0#Base64Binary">9iOMGex3ZDEpdw4xsAweWA==</wsse:Nonce>
    <wsu:Created>2016-01-12T19:37:59.328Z</wsu:Created>
  < /wsse:UsernameToken>
</wsse:Security>
<soapenv:Header/>
<soapenv:Body>
  <emp:setEmpsV4>
    <concept>111</concept>
    <storeNum>AB123</storeNum>
    <emps>
      <!--Zero or more repetitions:-->
      <item>
        <!--Optional:-->
        <address>Test Address</address>
        <!--Optional:-->
        <address2>Test Address 2</address2>
        <!--Optional:-->
        <altId>10</altId>
        <!--Optional:-->
        <city>Austin</city>
        <clientId>123456789</clientId>
        <!--Optional:-->
        <email>test@email.com</email>
        <empNum>100</empNum>
        <!--Optional:-->
        <FName>Sandra</FName>
        <hsId>-1</hsId>
        <!--Optional:-->
        <LName>White</LName>
        <!--Optional:-->
        <mobile>(405) 567-2567</mobile>
        <!--Optional:-->
        <NName>pref name Sandra</NName>
        <!--Optional:-->
        <phone>(843) 883-4228</phone>
        <!--Optional:-->
        <state>TEXAS</state>
        <status>1</status>
        <!--Optional:-->
        <zipCode>75104</zipCode>
      </item>
    </emps>
  </emp:setEmpsV4>
</soapenv:Body>
</soapenv:Header>
</wsse:Security>
</wsse:UsernameToken>
</wsse:PasswordText>
</wsse:Nonce>
</wsu:Created>
</wsse:Security>
</soapenv:Header>
</soapenv:Body>
</emp:setEmpsV4>
</concept>
</storeNum>
</emps>
</item>
</address>
</address2>
</altId>
</city>
</clientId>
</email>
</empNum>
</FName>
</hsId>
</LName>
</mobile>
</NName>
</phone>
</state>
</status>
</zipCode>

```

```

<!--Optional:-->
<alphanumericHrId>10alpha</alphanumericHrId>
<!--Optional:-->      <dob>1988-04-01T00:00:00</dob>
<!--Optional:-->
<hireDate>2021-02-04T00:00:00</hireDate>
<!--Optional:-->
<termDate>2024-02-04T00:00:00</termDate>
<!--Optional:-->
<customFields>
  <!--Zero or more repetitions:-->
  <customField name="customField" value="anyValue"/>
</customFields>
<!--Optional:-->
<homeStore>true</homeStore>
<!--Optional:-->
<payType>salaried</payType>
<!--Optional:-->
<otExempt>true</otExempt>
<!--Optional:-->
<salaryAmount>15000000</salaryAmount>
<!--Optional:-->
<salaryFrequency>4</salaryFrequency>
</item>
</emps>
</emp:setEmpsV4>
</soapenv:Body>
</soapenv:Envelope>

```

RESPONSE:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmpsV4Response
xmlns:ns1="http://services.hotschedules.com/api/services/EmpService">
      <return>
        <failCount>0</failCount>
        <success>true</success>
        <successCount>1</successCount>
      </return>
    </ns1:setEmpsV4Response>
  </soap:Body>
</soap:Envelope>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client does not have permission to use the EmpService operation: {}
    </faultstring>
      <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/EmpService"/>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

The following errors can be returned in the faultstring element:

faultstring	Description
client does not have permission to use the EmpService Set Employees operation {}	Insufficient permissions to use the API. Please contact Fourth to allow access.
client not found for concept: {} store: {}	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
Edit Employee Pay Type permission is not enabled for client	Insufficient permissions to use the API. Please contact Fourth to allow access.
Found invalid Pay Type value: {}. Only 'hourly' and 'salaried' Pay Types are allowed.	Invalid Pay Type. Please ensure that the params are correct and resubmit the request.
Employee Management is not enabled for client	Insufficient permissions to use the API. Please contact Fourth to allow access.
Cannot identify employee {} /Employee {} is not found	Employee not found, please check that employee exists and correctly mapped by pos id or hrid. Please ensure that the params are correct and resubmit the request.
{ } is not a valid Employee Type.	Wrong employee type in request, check payType in request (hourly/salaried). Please ensure that the params are correct and resubmit the request.

Partially successful responses

The API will attempt to process all the employee information that pass validation. Invalid employee items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of <emps> items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmpsV3Response xmlns:ns1="http://services.hotschedules.com/api/services/EmpService ">
      <return>
        <errors> error parsing employee: {}</errors>
        <failCount>1</failCount>
        <success>false</success>
        <successCount>0</successCount>
      </return>
    </ns1: setEmpsV3Response>
  </soap:Body>
</soap:Envelope>
```

Validation error	Description
error parsing employee: {}	Please ensure that the params are correct and resubmit the request. Contact Fourth support
fatal error in employee list insert/update: {}	Please ensure that the params are correct and resubmit the request. Contact Fourth support

setSalariedJobs

This method takes in a concept ID, store ID, and an array of EmpJob objects to assign salaried pay rates to individual employees via their primary job.

Call:

Name	Type	Required	Description/Example
storeNum	int	True	Store ID

concept	int	True	Concept ID - Provided by Fourth
posEmpId	int	True	The employee POS ID
posJobId	int	True	The ID primary job of the employee
annualSalary	int	True	The annual salary of the employee

Response:

Name	Type	Description/Example
posEmpId	int	The employee POS ID
posJobId	int	The ID primary job of the employee
annualSalary	int	The annual salary of the employee (In the below example, the annual salary is 50,000.00 or 50k)

Example setSalariedJobs call:

```

<concept>1234</concept>
<storeNum>1</storeNum>
<jobs>
  <item>
    <annualSalary>5000000</annualSalary>
    <posEmpId>333</posEmpId>
    <posJobId>10</posJobId>
  </item>
</jobs>

```

setAverageWage

This method that belongs to EmpService, updates the client's average wage through either the XR or XR_ALPHA, depending on the value of the company setting API_USER_ACCESS_ALPHA_EXTERNAL_REF.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept XR - Provided by Fourth.
clientData	Array<Item>	True	List of items. Occurrences [1..n]
item	Object	True	Object containing locNum & averageWage
item.averageWage	BigDecimal	True	The average wage value to set to the client
item.locNum	String	True	Client XR Alpha if client uses API_USER_ACCESS_ALPHA_EXTERNAL_REF setting. Client XR if not.

Response:

Name	Type	Description/Example
errors	List<Error>	List of error. Occurrences [0..n]
error	Object	Object contains locNum and reason
error.locNum	String	Received locNum
error.reason	String	Reason of the error
failCount	Int	Amount of errors
success	Boolean	Indicates if at least 1 record updated successfully
successCount	Int	Amount of successfully updated records

Example setAverageWage call:

```
<concept>0</concept>
<clientData>
  <!--Zero or more repetitions:-->
  <item>
    <averageWage>140.56</averageWage>
    <locNum>1</locNum>
  </item>
  <item>
    <averageWage>124.5</averageWage>
    <locNum>ABC123</locNum>
  </item>
</clientData>
```

ScheduleService Available Methods

GetLocations

This method takes in a concept and store number, then returns a list of all the locations in that store.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID

Response:

Name	Type	Description/Example
disabled	Boolean	Location is disabled:

		• True • False
id	Int	ID of location
name	String	Name of location

Example getLocations call:

```
<item>
  <disabled>>false</disabled>
  <id>787113532</id>
  <name>Patio 1</name>
</item>
```

getSchedule

This method takes in a concept, store number, start and end date and returns the schedule data within the specified range.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Store ID
start	String	True	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
end	String	True	End date and time of requested range (ex: 2015-10-10T00:00:00-05:00)

Response:

Name	Type	Description/Example
empHSId	Int	Internal Fourth Employee ID
empPosId	Int	Employee POS ID
jobHSId	Int	Internal Fourth Job ID
jobPosId	Int	Job POS ID
in	String	Start date and time of shift
out	String	End date and time of shift
weekEnd	String	End date of work week.
weekStart	String	Start date of work week

Example getSchedule response:

```

<item>
  <empHSId>3028402</empHSId>
  <empPosId>1057</empPosId>
  <jobHsId>786323136</jobHsId>
  <jobPosId>22</jobPosId>
  <in>2019-10-19T05:00:00-05:00</in>
  <out>2019-10-19T09:30:00-05:00</out>
  <weekEnd>2019-10-20T00:00:00-05:00</weekEnd>
  <weekStart>2019-10-14T00:00:00-05:00</weekStart>
</item>
<item>
  <empHSId>3010271</empHSId>
  <empPosId>1036</empPosId>
  <jobHsId>786323131</jobHsId>
  <jobPosId>12</jobPosId>
  <in>2019-10-19T05:00:00-05:00</in>
  <out>2019-10-19T12:30:00-05:00</out>
  <weekEnd>2019-10-20T00:00:00-05:00</weekEnd>
  <weekStart>2019-10-14T00:00:00-05:00</weekStart>
</item>

```

getScheduleV2

This method takes in a concept, store number, start and end date and returns the schedule data within the specified range.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Store ID
start	String	True	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
end	String	True	End date and time of requested range (ex: 2015-10-10T00:00:00-05:00)

Response:

Name	Type	Description/Example
empHSId	Int	Internal Fourth Employee ID
empPosId	Int	Employee POS ID
jobHsId	Int	Internal Fourth Job ID
jobPosId	Int	Job POS ID

in	String	Start date and time of shift
out	String	End date and time of shift
weekEnd	String	End date of work week.
weekStart	String	Start date of work week
locationId	Int	Location ID
ovtMinutes	Int	Scheduled overtime minutes
ovtRate	Float	Overtime pay rate
payRate	Float	Regular pay rate
regMinutes	Int	Scheduled regular minutes
scheduleId	Int	Internal schedule ID
specialPay	Float	Special Pay Rate

Example getScheduleV2 call:

```

<item>
  <empHSId>3010265</empHSId>
  <empPosId>1030</empPosId>
  <jobHsId>786323136</jobHsId>
  <jobPosId>22</jobPosId>
  <in>2019-10-20T05:00:00-05:00</in>
  <out>2019-10-20T09:30:00-05:00</out>
  <weekEnd>2019-10-20T00:00:00-05:00</weekEnd>
  <weekStart>2019-10-14T00:00:00-05:00</weekStart>
  <locationId>-1</locationId>
  <ovtMinutes>0</ovtMinutes>
  <ovtRate>0.0</ovtRate>
  <payRate>9.0</payRate>
  <regMinutes>270</regMinutes>
  <scheduleId>781691539</scheduleId>
  <specialPay>0.0</specialPay>
</item>

```

getScheduleV3

This method takes in a concept, store number, start and end date and returns the schedule data within the specified range.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Store ID

start	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
empHSId	Int	Internal Employee Fourth ID
empPosId	Int	Employee POS ID
jobHsId	Int	Internal Job Fourth ID
jobPosId	Int	Job POS ID
inDate	Array	- Day (Int) - Month (Int) - Year (Int)
inTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
locationId	String	ID of location
outDate	Array	- Day (Int) - Month (Int) - Year (Int)
outTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
ovtMinutes	Int	Scheduled overtime minutes
ovtRate	Float	Overtime pay rate
payRate	Float	Regular pay rate
regMinutes	Int	Scheduled regular minutes
scheduleId	Int	ID of schedule
specialPay	Float	Special Pay Rate
weekEnd	Array	- Day (Int) - Month (Int) - Year (Int)
weekStart	Array	- Day (Int) - Month (Int) - Year (Int)

Example getScheduleV3 call:

```
<item>
  <empHSId>3010265</empHSId>
  <empPosId>1030</empPosId>
  <jobHsId>786323136</jobHsId>
  <jobPosId>22</jobPosId>      <inDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </inDate>
  <inTime>
    <hours>5</hours>
    <militaryTime>true</militaryTime>
    <minutes>0</minutes>
<seconds>0</seconds>
  </inTime>
  <locationId>-1</locationId>
  <outDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </outDate>
  <outTime>
    <hours>9</hours>
    <militaryTime>true</militaryTime>
    <minutes>30</minutes>
    <seconds>0</seconds>
  </outTime>
  <ovtMinutes>0</ovtMinutes>
  <ovtRate>0.0</ovtRate>
  <payRate>9.0</payRate>
  <regMinutes>270</regMinutes>
  <scheduleId>781691539</scheduleId>
  <specialPay>0.0</specialPay>
  <weekEnd>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </weekEnd>
  <weekStart>
    <day>14</day>
    <month>10</month>
    <year>2019</year>
```

</weekStart>
</item>

getScheduleV4

This method takes in a concept, store number, start and end date and returns the schedule data within the specified range.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Store ID
start	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
empHSId	Int	Internal Employee Fourth ID
empPosId	Int	Employee POS ID
jobHSId	Int	Internal Job Fourth ID
jobPosId	Int	Job POS ID
inDate	Array	- Day (Int) - Month (Int) - Year (Int)
inTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
locationId	String	ID of location
outDate	Array	- Day (Int) - Month (Int) - Year (Int)
outTime	Array	Hours (Int)
		- militaryTime (Boolean) - Minutes (Int) - Seconds (Int)

ovtMinutes	Int	Scheduled overtime minutes
ovtRate	Float	Overtime pay rate
payRate	Float	Regular pay rate
regMinutes	Int	Scheduled regular minutes
scheduleId	Int	ID of schedule
specialPay	Float	Special Pay Rate
weekEnd	Array	• Day (Int) • Month (Int) • Year (Int)
weekStart	Array	• Day (Int) • Month (Int) • Year (Int)
breakMinutes	Int	Total minutes on break
mealMinutes	Int	Total minutes on meal

Example getScheduleV4 call:

```

<item>
  <empHSId>3010265</empHSId>
  <empPosId>1030</empPosId>
  <jobHsId>786323136</jobHsId>
  <jobPosId>22</jobPosId>
  <inDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </inDate>
  <inTime>
    <hours>5</hours>
    <militaryTime>true</militaryTime>
    <minutes>0</minutes>
<seconds>0</seconds>
  </inTime>
  <locationId>-1</locationId>
  <outDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </outDate>
  <outTime>
    <hours>9</hours>
    <militaryTime>true</militaryTime>
    <minutes>30</minutes>

```

```

        <seconds>0</seconds>
    </outTime>
    <ovtMinutes>0</ovtMinutes>
    <ovtRate>0.0</ovtRate>
    <payRate>9.0</payRate>
    <regMinutes>270</regMinutes>
    <scheduleId>781691539</scheduleId>
    <specialPay>0.0</specialPay>
    <weekEnd>
        <day>20</day>
        <month>10</month>
        <year>2019</year>
    </weekEnd>
    <weekStart>
        <day>14</day>
        <month>10</month>
        <year>2019</year>
    </weekStart>
    <breakMinutes>10</breakMinutes>
    <mealMinutes>0</mealMinutes>
</item>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client does not have permission to use the ScheduleService operation: {}
    </faultstring>
    <detail>
      <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/ScheduleService"/>
    </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element.

faultstring	Description
client does not have permission to use the ScheduleService operation: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.

getScheduleV5

This method takes in a concept, alphanumeric store number, start and end date and returns the schedule data within the specified range.

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Alphanumeric Store ID
start	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	True	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
empHSId	Int	Internal Employee Fourth ID
empPosId	Int	Employee POS ID
shiftId	String	Shift id
jobHsId	Int	Internal Job Fourth ID
jobPosId	Int	Job POS ID
inDate	Array	- Day (Int) - Month (Int) - Year (Int)
inTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
locationId	String	ID of location
outDate	Array	- Day (Int) - Month (Int) - Year (Int)
outTime	Array	Hours (Int)
		- militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
ovtMinutes	Int	Scheduled overtime minutes
ovtRate	Float	Overtime pay rate
payRate	Float	Regular pay rate

regMinutes	Int	Scheduled regular minutes
scheduleId	Int	ID of schedule
specialPay	Float	Special Pay Rate
weekEnd	Array	• Day (Int) • Month (Int) • Year (Int)
weekStart	Array	• Day (Int) • Month (Int) • Year (Int)
breakMinutes	Int	Total minutes on break
mealMinutes	Int	Total minutes on meal

Example getScheduleV5 response:

```

<item>
  <empHSId>3010265</empHSId>
  <empPosId>1030</empPosId>
  <shiftId>1030</shiftId>
  <jobHsId>786323136</jobHsId>
<jobPosId>22</jobPosId>
  <inDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </inDate>
  <inTime>
    <hours>5</hours>
    <militaryTime>true</militaryTime>
    <minutes>0</minutes>
    <seconds>0</seconds>
  </inTime>
  <locationId>-1</locationId>
  <outDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </outDate>
  <outTime>
    <hours>9</hours>
    <militaryTime>true</militaryTime>
    <minutes>30</minutes>
    <seconds>0</seconds>
  </outTime>

```

```

<ovtMinutes>0</ovtMinutes>
<ovtRate>0.0</ovtRate>
<payRate>9.0</payRate>
<regMinutes>270</regMinutes>
<scheduleId>781691539</scheduleId>
<specialPay>0.0</specialPay>
<weekEnd>
  <day>20</day>
  <month>10</month>
  <year>2019</year>
</weekEnd>
<weekStart>
  <day>14</day>
  <month>10</month>
  <year>2019</year>
</weekStart>
<breakMinutes>10</breakMinutes>
<mealMinutes>0</mealMinutes>
</item>

```

getScheduleStatus

This method takes in a concept and group ID, and returns various schedule information for all stores within the specified group

Call:

Name	Type	Required	Description/Example
concept	Int	True	Concept ID - Provided by Fourth
groupNum	Int	True	Group External Ref ID - Provided by Fourth

Response:

Name	Type	Description/Example
edited	Boolean	False = Schedule has not been edited True = Schedule has been edited
posted	Boolean	False = Schedule is not posted True = Schedule is posted
postedByFName	String	First name of employee posting schedule
postedByHRID	Int	HR ID of employee posting schedule
postedByLName	String	Last name of employee posting schedule
postedByPosID	Int	POS ID of employee posting schedule

postedDateTime	String	When schedule was posted
scheduleId	Int	ID of schedule
scheduleName	String	Name of schedule
storeNumber	Int	Store ID

Example getScheduleStatus response:

```

<item>
  <edited>>false</edited>
  <posted>true</posted>
  <postedByFName>Dean</postedByFName>
  <postedByHRID>-1</postedByHRID>
  <postedByLName>Manager</postedByLName>
  <postedByPosID>1070</postedByPosID>
  <postedDateTime>2019-10-11T13:30:36.007-05:00</postedDateTime>
  <scheduleId>781691492</scheduleId>
  <scheduleName>Server</scheduleName>
  <storeNumber>2</storeNumber>
</item>

```

getShiftsV3

This method takes in a concept ID, store ID, start and end dates and three flags (isHouse, isScheduled and isPosted). All 3 flags need to be included for each call:

- **isHouse** - includes scheduled or posted shifts that are not assigned to an employee (called 'house shifts' in Fourth).
- **isScheduled** - includes shifts that are in schedules that have been saved in Fourth, but might not have been posted.
- **isPosted** - includes shifts that are in schedules that have been set to the 'posted' status within Fourth.

This method returns extended scheduled shift data, including location ID, regular pay rate, OT pay rate, scheduled regular minutes, scheduled OT minutes (if any), scheduled special pay (if any) and the unique schedule ID, internal to HS.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	• Day (Int) - 1-30 or 31 • Month (Int) - 1-12 • Year (Int) - YYYY, Ex: 2019

end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
isHouse	Boolean	true	Include house shifts? These are shifts that exist, but are not assigned to an employee. Note: If isScheduled is set to "true," isHouse needs to be set to "false."
isScheduled	Boolean	true	Include scheduled shifts? These are shifts which are on a schedule that has been saved in Fourth, but might not have been posted
isPosted	Boolean	true	These are shifts that are in schedules that have been set to the 'posted' status within Fourth.
jobCodes	IntArray	true	Integer array of job codes to be included in this method's return. if this parameter is null or empty, all jobs will be included.

Response:

Name	Type	Description/Example
empHsId	Int	Fourth internal employee account ID
empPosId	Int	POS Numeric Employee ID
jobHsId	Int	Fourth internal job code ID
jobPosId	Int	POS job code ID
inDate	Array	- Day (Int) - Month (Int) - Year (Int)
inTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
locationId	Int	Location ID
outDate	Array	- Day (Int) - Month (Int) - Year (Int)
outTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
ovtMinutes	Int	Total Overtime minutes for employee

ovtRate	Float	Overtime pay rate
payRate	Float	Regular pay rate
regMinutes	Int	Total regular minutes for employee
scheduleId	Int	Internal Schedule ID
specialPay	Float	Special time pay rate
weekEnd	Array	Business week end date • Day (Int) • Month (Int) • Year (Int)
weekStart	Array	Business week start date • Day (Int) • Month (Int) • Year (Int)

Example getShiftsV3 response:

```

<item>
  <empHSId>3028756</empHSId>
  <empPosId>1068</empPosId>
  <jobHsId>786323136</jobHsId>
  <jobPosId>22</jobPosId>
  <inDate>
    <day>8</day>
    <month>10</month>
    <year>2019</year>
  </inDate>
  <inTime>
    <hours>5</hours>
    <militaryTime>true</militaryTime>
    <minutes>0</minutes>
    <seconds>0</seconds>
  </inTime>
  <locationId>-1</locationId>
  <outDate>
    <day>8</day>
    <month>10</month>
    <year>2019</year>
  </outDate>
  <outTime>
    <hours>9</hours>
    <militaryTime>true</militaryTime>
    <minutes>30</minutes>
    <seconds>0</seconds>

```

```

</outTime>
<ovtMinutes>0</ovtMinutes>
<ovtRate>0.0</ovtRate>
<payRate>9.0</payRate>
<regMinutes>270</regMinutes>
<scheduleId>781691539</scheduleId>
<specialPay>0.0</specialPay>
<weekEnd>
  <day>13</day>
  <month>10</month>
  <year>2019</year>
</weekEnd>
<weekStart>
  <day>7</day>
  <month>10</month>
  <year>2019</year>
</weekStart>
</item>

```

TimeCardService Available Methods

getTimeCards

This method takes in a concept ID, store ID, start and end dates. It returns an array of wsTimeCard3 objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in and clock out times. If this store is using Fourth' web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	• Day (Int) • Month (Int) • Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out

Example getTimeCards response:

```

<item>
  <breakMinutes>0</breakMinutes>
  <empPosId>1028</empPosId>
  <extId>2146230310858</extId>
  <hsId>2146230310916</hsId>
  <jobExtId>13</jobExtId>
  <jobId>-1</jobId>
  <jobName>Host</jobName>
  <ovtHrs>0.0</ovtHrs>
  <ovtMins>0</ovtMins>
  <ovtTtl>0.0</ovtTtl>
  <ovtWage>10.5</ovtWage>
  <regHrs>4.8</regHrs>
  <regTtl>0.0</regTtl>
  <regWage>0.0</regWage>
  <spcHrs>0.0</spcHrs>
  <spcTtl>7.25</spcTtl>

```

```
<storeNum>2</storeNum>
<businessDate>
  <day>20</day>
  <month>10</month>
  <year>2019</year>
</businessDate>
<clockIn>2019-10-20T11:07:00-05:00</clockIn>
<clockOut>2019-10-20T15:55:00-05:00</clockOut>
</item>
```

getTimeCardsDeclaredTips

This method takes in a concept ID, store ID, start and end dates. It returns an array of wsTimeCard3 objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in, clock out times and declared tips. If this store is using Fourth' web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	Day (Int)

		• Month (Int) • Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out
declaredTips	Float	Amount of Declared tips

Example getTimeCardsDeclaredTips response: <item>

```

    <breakMinutes>0</breakMinutes>
    <empPosId>1028</empPosId>
    <extId>2146230310858</extId>
    <hsId>2146230310916</hsId>
    <jobExtId>13</jobExtId>
    <jobId>-1</jobId>
    <jobName>Host</jobName>
    <ovtHrs>0.0</ovtHrs>
    <ovtMins>0</ovtMins>
    <ovtTtl>0.0</ovtTtl>
    <ovtWage>10.5</ovtWage>
    <regHrs>4.8</regHrs>
    <regTtl>0.0</regTtl>
    <regWage>0.0</regWage>
    <spcHrs>0.0</spcHrs>
    <spcTtl>7.25</spcTtl>
    <storeNum>2</storeNum>
    <businessDate>
      <day>20</day>
      <month>10</month>
      <year>2019</year>
    </businessDate>
    <clockIn>2019-10-20T11:07:00-05:00</clockIn>
    <clockOut>2019-10-20T15:55:00-05:00</clockOut>
    <declaredTips>0.0</declaredTips>
  </item>

```

getTimeCardsV2

This method takes in a concept ID, store ID, start and end dates. It returns an array of wsTimeCard3 objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in and clock out times. If this store is using Fourth' web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	- Day (Int) - Month (Int) - Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out
dtHrs	Float	Double time hours worked
dtMins	Float	Double time minutes worked
dtTtl	Float	Total amount of wages for double time worked.
dtWage	Float	Double time wage.

Example getTimeCardsV2 response:


```

<item>
  <breakMinutes>0</breakMinutes>
  <empPosId>1045</empPosId>
  <extId>2146230310855</extId>
  <hsId>2146230310899</hsId>
  <jobExtId>31</jobExtId>
  <jobId>964288574</jobId>
  <jobName>Cook</jobName>
  <ovtHrs>0.0</ovtHrs>
  <ovtMins>0</ovtMins>
  <ovtTtl>0.0</ovtTtl>
  <ovtWage>13.5</ovtWage>
  <regHrs>5.5</regHrs>
  <regTtl>49.5</regTtl>
  <regWage>0.0</regWage>
  <spcHrs>0.0</spcHrs>
  <spcTtl>9.0</spcTtl>
  <storeNum>2</storeNum>
  <businessDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </businessDate>
  <clockIn>2019-10-20T08:28:00-05:00</clockIn>
  <clockOut>2019-10-20T13:58:00-05:00</clockOut>
  <dtHrs>0.0</dtHrs>
  <dtMins>0</dtMins>
  <dtTtl>0.0</dtTtl>
  <dtWage>18.0</dtWage>
</item>

```

getTimeCardsV3

This method takes in a concept ID, store ID, start and end dates. It returns an array of wsTimeCard3 objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in and clock out times. If this store is using Fourth' web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
------	------	----------	---------------------

concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	- Day (Int) - Month (Int) - Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out
empHsId	Int	Internal Fourth ID

Example getTimeCardsV3 response:

```

<item>
  <breakMinutes>0</breakMinutes>
  <empPosId>1028</empPosId>
  <extId>2146230310858</extId>

```

```

<hsId>2146230310916</hsId>
<jobExtId>13</jobExtId>
<jobId>786323130</jobId>
<jobName>Host</jobName>
<ovtHrs>0.0</ovtHrs>
<ovtMins>0</ovtMins>
<ovtTtl>0.0</ovtTtl>
<ovtWage>10.5</ovtWage>
<regHrs>4.8</regHrs>
<regTtl>0.0</regTtl>
<regWage>0.0</regWage>
<spcHrs>0.0</spcHrs>
<spcTtl>7.25</spcTtl>
<storeNum>2</storeNum>
<businessDate>
  <day>20</day>
  <month>10</month>
  <year>2019</year>
</businessDate>
<clockIn>2019-10-20T11:07:00-05:00</clockIn>
<clockOut>2019-10-20T15:55:00-05:00</clockOut>
<empHsId>3010262</empHsId>
</item>

```

getTimeCardsV4Beta

This method takes in a concept ID, store ID, start and end dates. It returns an array of objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in and clock out times. If this store is using Fourth's web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	• Day (Int) • Month (Int) • Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out
dtHrs	Float	Double time hours worked
dtMins	Float	Double time minutes worked
dtTtle	Float	Total amount of wages for double time worked.
dtWage	Float	Double time wage.
autoClockedOut	Boolean	Was this time card auto clocked out?
cashTips	Float	Amount of cash tips
ccTips	Float	Amount of credit card tips
empHsId	Int	Internal Fourth Employee ID
exceptions	Array	• Type (String) • Value (Float)
punchAcknowledged	Boolean	Has punch been acknowledged?
punchLocked	Boolean	Has punch been locked?
punchReviewed	Boolean	Has punch been reviewed?
serviceCharge	Float	Amount of service charge
tipPool	Float	Amount in tip pool

Example getTimeCardsV4Beta response:

```
<item>
  <breakMinutes>0</breakMinutes>
  <empPosId>1052</empPosId>
  <extId>2146230310845</extId>
  <hsId>2146230310887</hsId>
  <jobExtId>22</jobExtId>
  <jobId>786323136</jobId>
  <jobName>Prep</jobName>
  <ovtHrs>0.0</ovtHrs>
  <ovtMins>0</ovtMins>
  <ovtTtl>0.0</ovtTtl>
  <ovtWage>13.5</ovtWage>
  <regHrs>8.14</regHrs>
  <regTtl>73.08</regTtl>
  <regWage>0.0</regWage>
  <spcHrs>0.0</spcHrs>
  <spcTtl>9.0</spcTtl>
  <storeNum>2</storeNum>
  <businessDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </businessDate>
  <clockIn>2019-10-20T04:52:00-05:00</clockIn>
  <clockOut>2019-10-20T12:59:00-05:00</clockOut>
  <dtHrs>0.0</dtHrs>
  <dtMins>0</dtMins>
  <dtTtl>0.0</dtTtl>
  <dtWage>18.0</dtWage>
  <autoClockedOut>>false</autoClockedOut>
  <breaks/>
  <cashTips>0.0</cashTips>
  <ccTips>0.0</ccTips>
  <empHsId>3010680</empHsId>
  <exceptions>
    <exception>
      <type>noBreaksTaken</type>
      <value>0.0</value>
    </exception>
    <exception>
      <type>noBreaksTaken</type>
      <value>0.0</value>
    </exception>
  </exceptions>
</item>
```

```

    </exception>
    <exception>
      <type>DailyMaxDuration</type>
      <value>0.14</value>
    </exception>
    <exception>
      <type>noMealsTaken</type>
      <value>0.0</value>
    </exception>
  </exceptions>
  <punchAcknowledged>false</punchAcknowledged>
  <punchLocked>false</punchLocked>
  <punchReviewed>false</punchReviewed>
  <serviceCharge>0.0</serviceCharge>
  <tipPool>0.0</tipPool>
</item>

```

getTimeCardsV5

The GET Timecards API v5 introduces expanded information, enabling users to retrieve additional employee's timecard data compared to previous versions of the API. It now includes critical information such as Pay Exceptions, PTOs, and Individual Acknowledgements.

getTimecardsV5 is based in getTimecardsV4Beta and uses the same request call.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID
storeNum	Int	true	Store ID
start	Array	true	Day (Int) - 1-30 or 31 Month (Int) - 1-12 Year (Int) - YYYY, Ex: 2019
end	Array	true	Day (Int) - 1-30 or 31 Month (Int) - 1-12 Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Long	Total break minutes
empPosId	Int	Employee POS ID
extId	Long	Timecard External ID
hrId	String	Employee hr ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	String	Store ID
businessDate	Array	• day (Int) • month (Int) • year (Int)
clockIn	DateTime	Time employee clocked in Using the following format: yyyy-MM-dd'T'HH:mm:ss.SSSZ (where "Z" is time zone offset from UTC)
clockOut	DateTime	Time employee clocked out Using the following format: yyyy-MM-dd'T'HH:mm:ss.SSSZ (where "Z" is time zone offset from UTC)
breaks	Array	• breakEnd (DateTime, using the same format as clockIn/clockOut) • breakStart (DateTime, using the same format as clockIn/clockOut)

		breakType (String, supported types are: "BREAK", "MEAL")
dtHrs	Float	Double time hours worked
dtTtl	Float	Total amount of wages for double time worked.
dtWage	Float	Double time wage.
autoClockedOut	Boolean	Was this time card auto clocked out?
cashTips	Float	Amount of cash tips
ccTips	Float	Amount of credit card tips
exceptions	Array	- Type (String) - Value (Float)
punchAcknowledged	String	"Acknowledged" + datetime of acknowledgement if it was acknowledged "False" if it wasn't acknowledged "Disagree" + employee comment + datetime of acknowledgement if employee disagreed "Agree" + datetime of acknowledgement if employee agreed
punchLocked	Boolean	Has punch been locked?
punchReviewed	Boolean	Has punch been reviewed?
punchReviewDateTime	DateTime	Time of the punch review if it was reviewed
serviceCharge	Float	Amount of service charge
tipPool	Float	Amount in tip pool
payType	String	Supported Pay Types: "HOURLY", "SALARIED"
payExceptions	Array	- empPosId (Int) - hrId (Long) - earningCode (String) Default values "SS" = Split Shift "RTR" = Right to Rest "M" = Meal Penalty "BR" = Break Penalty "MT" = Minimum Time "PRM" = Premium Pay "PP" = Predictability Pay

		• hrs (Float) • ttl (Float) • timeCardExternalId (Long)
PTO	Array	• empPosId (Int) • hrId (Long) • date (String) • payType (String) • ptoType (String) • ptoHrs (Float) • hourlyRate (Float)
tipShortfalls	Array	• empPosId (Int) • hrId (Long) • workWeekTSF (Array) - o workWeekStartDate (String) - o calculationStartDate (String) - o calculationEndDate (String) - o totalTSF (Float)

Example getTimeCardsV5 response:

```

<timeCards>
  <timeCard>
    <breakMinutes>9</breakMinutes>
    <empPosId>7</empPosId>
    <extId>2364512240337</extId>
    <hrId>123</hrId>
    <jobExtId>123</jobExtId>
    <jobId>1091454667</jobId>
    <jobName>Cook 10</jobName>
    <ovtHrs>0.0</ovtHrs>
    <ovtTtl>0.0</ovtTtl>
    <ovtWage>15.0</ovtWage>
    <regHrs>7.0</regHrs>
    <regTtl>70.0</regTtl>
    <regWage>10.0</regWage>
    <spcHrs>0.0</spcHrs>
    <spcTtl>0.0</spcTtl>
    <storeNum>2</storeNum>
    <businessDate>
      <day>15</day>
    
```

```
<month>7</month>
<year>2024</year>
</businessDate>
<clockIn>2024-07-16T01:00:00.000-05:00</clockIn>
<clockOut>2024-07-16T08:00:00.000-05:00</clockOut>
<dtHrs>0.0</dtHrs>
<dtTtl>0.0</dtTtl>
<dtWage>20.0</dtWage>
<autoClockedOut>>false</autoClockedOut>
<breaks>
  <break>
    <breakEnd>2024-07-23T11:05:11.000-05:00</breakEnd>
    <breakStart>2024-07-23T11:05:05.000-05:00</breakStart>
    <breakType>BREAK</breakType>
  </break>
  <break>
    <breakEnd>2024-07-23T11:05:21.000-05:00</breakEnd>
    <breakStart>2024-07-23T11:05:18.000-05:00</breakStart>
    <breakType>MEAL</breakType>
  </break>
</breaks>
<cashTips>2.0</cashTips>
<ccTips>3.0</ccTips>
<exceptions>
  <exception>
    <type>Break</type>
    <value>7.0</value>
  </exception>
  <exception>
    <type>Meal</type>
    <value>7.0</value>
  </exception>
  <exception>
    <type>DailyMinDuration</type>
    <value>7.0</value>
  </exception>
  <exception>
    <type>DailyMaxDuration</type>
    <value>7.0</value>
  </exception>
  <exception>
    <type>WeeklyMinDuration</type>
    <value>7.0</value>
  </exception>
</exceptions>
```

```

    </exception>
    <exception>
      <type>WeeklyMaxDuration</type>
      <value>7.0</value>
    </exception>
    <exception>
      <type>DailyActVsSched</type>
      <value>7.0</value>
    </exception>
    <exception>
      <type>WeeklyActVsSched</type>
      <value>7.0</value>
    </exception>
    <exception>
      <type>AutoClockOut</type>
      <value>2.0</value>
    </exception>
    <exception>
      <type>Overtime</type>
      <value>7.0</value>
    </exception>
    <exception>
      <type>OverlappingPunch</type>
      <value>7.0</value>
    </exception>
  </exceptions>
  <punchAcknowledged>Disagree - I worked as a server not a bartender 2024-06-20
11:48:00</punchAcknowledged>
  <punchLocked>>false</punchLocked>
  <punchReviewed>>false</punchReviewed>
  <serviceCharge>4.0</serviceCharge>
  <tipPool>5.0</tipPool>
  <payType>hourly</payType>
</timeCard>
</timeCards>
<payExceptions>
  <payException>
    <empPosId>7</empPosId>
    <hrId>123</hrId>
    <earningCode>PRM</earningCode>
    <hrs>1.5</hrs>
    <ttl>15.3</ttl>
    <timeCardExternalId>2364319846335</timeCardExternalId>

```

```

</payException>
<payException>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <earningCode>RTR</earningCode>
  <hrs>1</hrs>
  <ttd>10</ttd>
  <timeCardExternalId>2364319846335</timeCardExternalId>
</payException>
<payException>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <earningCode>PP</earningCode>
  <hrs>0</hrs>
  <ttd>20</ttd>
  <timeCardExternalId>2364319846335</timeCardExternalId>
</payException>
<payException>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <earningCode>MT</earningCode>
  <hrs>1</hrs>
  <ttd>10</ttd>
  <timeCardExternalId>2364319846335</timeCardExternalId>
</payException>
<payException>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <earningCode>SS</earningCode>
  <hrs>0</hrs>
  <ttd>6</ttd>
  <timeCardExternalId>2364319846335</timeCardExternalId>
</payException>
<payException>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <earningCode>MBP</earningCode>
  <hrs>1.5</hrs>
  <ttd>15.3</ttd>
  <timeCardExternalId>2364319846335</timeCardExternalId>
</payException>
</payExceptions>
<ptos>

```

```

<pto>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <date> 2024-09-02 00:00:00.000 </date>
  <ptoType>Vacation</ptoType>
  <ptoHrs>8</ptoHrs>
  <hourlyRate>10</hourlyRate>
</pto>
<pto>
  <empPosId>7</empPosId>
  <hrId>123</hrId>
  <date> 2024-09-03 00:00:00.000 </date>
  <ptoType>Vacation</ptoType>
  <ptoHrs>8</ptoHrs>
  <hourlyRate>10</hourlyRate>
</pto>
</ptos>
<tipShortfalls>
  <employeeTSF>
    <empPosId>7</empPosId>
    <hrId>123</hrId>
    <workWeekTotals>
      <workWeekTSF>
        <workWeekStartDate>2024-07-20</workWeekStartDate>
        <calculationStartDate>2024-07-21</calculationStartDate>
        <calculationEndDate>2024-07-25</calculationEndDate>
        <totalTSF>225.00</totalTSF>
      </workWeekTSF>
    </workWeekTotals>
  </employeeTSF>
</tipShortfalls>

```

getTimeCardsV6

This method takes in a concept ID, alphanumeric store ID, start and end dates. It returns an array of wsTimeCard3 objects, which represent one employee time card each. Each time card has information for one employee punch record, including business date, regular and OT minutes and wages, clock in and clock out times. If this store is using Fourth' web-based timeclock for employee clock-in, any open punches in the date range are also included in the response.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	String	true	Alphanumeric Store ID
start	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
breakMinutes	Int	Total break minutes
empPosId	Int	Employee POS ID
extId	Int	Employee External ID
hsId	Int	Fourth Internal ID
jobExtId	Int	Job Code External ID
jobId	Int	Job Code POS ID
jobName	String	Job Code Name
ovtHrs	Float	Number of overtime hours worked
ovtMins	Float	Number of overtime minutes worked
ovtTtl	Float	Total amount of wages for overtime worked
ovtWage	Float	Overtime wage
regHrs	Float	Number of hours worked
regTtl	Float	Total amount of wages for regular hours worked
regWage	Float	Regular wage
spcHrs	Float	Number of special hours worked
spcTtl	Float	Total amount of wages for special hours worked
storeNum	Int	Store ID
businessDate	Array	- Day (Int) - Month (Int) - Year (Int)
clockIn	String	Time employee clocked in
clockOut	String	Time employee clocked out
dtHrs	Float	Double time hours worked
dtMins	Float	Double time minutes worked
dtTtl	Float	Total amount of wages for double time worked.
dtWage	Float	Double time wage.

Example getTimeCardsV6 response:

```

<item>
  <breakMinutes>0</breakMinutes>
  <empPosId>1045</empPosId>
  <extId>2146230310855</extId>
  <hsId>2146230310899</hsId>
  <jobExtId>31</jobExtId>
  <jobId>964288574</jobId>
  <jobName>Cook</jobName>
  <ovtHrs>0.0</ovtHrs>
  <ovtMins>0</ovtMins>
  <ovtTtl>0.0</ovtTtl>
  <ovtWage>13.5</ovtWage>
  <regHrs>5.5</regHrs>
  <regTtl>49.5</regTtl>
  <regWage>0.0</regWage>
  <spcHrs>0.0</spcHrs>
  <spcTtl>9.0</spcTtl>
  <storeNum>2</storeNum>
  <businessDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </businessDate>
  <clockIn>2019-10-20T08:28:00-05:00</clockIn>
  <clockOut>2019-10-20T13:58:00-05:00</clockOut>
  <dtHrs>0.0</dtHrs>
  <dtMins>0</dtMins>
  <dtTtl>0.0</dtTtl>
  <dtWage>18.0</dtWage>
</item>

```

setTimecards

This method takes in a concept id, store number, and a plethora of employee timecard information like wage rates, totals, start and end times to update time card information in Fourth for the given timeframe. ***This method does not work with Fourth Time & Attendance.**

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID

breakMinutes	Int	false	Total minutes on break
breakType	String	false	Meal or Break
empPosId	Int	true	Employee POS ID
extId	Int	false	External employee ID
hsId	Int	false	Fourth Internal ID
jobExtId	Int	false	External Job ID
jobId	Int	true	Job Code POS ID
jobName	String	false	Job Code Name
ovtHrs	Float	false	Number of overtime hours worked
ovtMins	Float	false	Number of overtime minutes worked
ovtTtl	Float	false	Total amount of wages for overtime worked
ovtWage	Float	false	Overtime wage
regHrs	Float	true	Number of hours worked
regTtl	Float	true	Total amount of wages for regular hours worked
regWage	Float	true	Regular wage
spcHrs	Float	false	Number of special hours worked
spcTtl	Float	false	Total amount of wages for special hours worked
storeNum	Int	true	Store ID
businessDate	String	true	Business date of timecards. (ex: 2020-0110T00:00:00Z)
clockIn	String	true	Date and time of clock in. (ex: 2020-0110T10:15:00Z)
clockOut	String	true	Date and time of clock out. (ex: 2020-0110T10:15:00Z)
start	String	true	Start date of time cards (ex: 2020-0110T00:00:00Z)
end	String	true	End date of time cards (ex: 2020-0111T00:00:00Z)

Example setTimecards call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<cards>
  <item>
    <breakMinutes>0</breakMinutes>
    <breakType>?</breakType>
    <empPosId>1</empPosId>
    <jobId>1</jobId>
    <jobName>Server</jobName>
    <ovtHrs>0</ovtHrs>
  
```



```

<ovtMins>0</ovtMins>
<ovtTtl>0</ovtTtl>
<ovtWage>0</ovtWage>
<regHrs>1</regHrs>
<regTtl>9</regTtl>
<regWage>9</regWage>
<spcHrs>0</spcHrs>
<spcTtl>0</spcTtl>
<storeNum>1100030</storeNum>
<businessDate>2020-01-10T00:00:00Z</businessDate>
<clockIn>2020-01-10T10:00:00Z</clockIn>
<clockOut>2020-01-10T11:00:00Z</clockOut>
</item>
</cards>
<start>2020-01-10T00:00:00Z</start>
<end>2020-01-11T00:00:00Z</end>

```

SetTimecardsDeclaredTips

This method takes in a concept id, store number, and a plethora of employee timecard information like wage rates, totals, start and end times to update time card information in Fourth for the given timeframe. ***This method does not work with Fourth Time & Attendance.**

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
breakMinutes	Int	false	Total minutes on break
breakType	String	false	Meal or Break
empPosId	Int	true	Employee POS ID
extId	Int	false	External employee ID
hsId	Int	false	Fourth Internal ID
jobExtId	Int	true	External Job ID - POS ID
jobId	Int	false	Internal Job ID - HS ID
jobName	String	false	Job Code Name
ovtHrs	Float	false	Number of overtime hours worked

ovtMins	Float	false	Number of overtime minutes worked
ovtTtl	Float	false	Total amount of wages for overtime worked
ovtWage	Float	false	Overtime wage
regHrs	Float	true	Number of hours worked
regTtl	Float	true	Total amount of wages for regular hours worked
regWage	Float	true	Regular wage
spcHrs	Float	false	Number of special hours worked
spcTtl	Float	false	Total amount of wages for special hours worked
storeNum	Int	true	Store ID
businessDate	Array	true	Business Date of timecards. • Day (Int) • Month (Int) • Year (Int)
clockIn	String	true	Date and time of clock in. (ex: 2020-0110T10:15:00Z)
clockOut	String	true	Date and time of clock out. (ex: 2020-0110T10:15:00Z)
declaredtips	float	false	Total of declared tips
start	Array	true	Start date of time cards • Day (Int) • Month (Int) • Year (Int)
end	Array	true	End date of time cards • Day (Int) • Month (Int) • Year (Int)

Example setTimecardsDeclaredTips call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<cards>
  <item>
    <empPosId>1</empPosId>
    <jobId>1</jobId>
    <regHrs>1</regHrs>
    <regTtl>9</regTtl>
    <regWage>9</regWage>
    <storeNum>1100030</storeNum>
  
```

```

    <businessDate>
      <day>12</day>
      <month>1</month>
      <year>2020</year>
    </businessDate>
    <clockIn>2020-01-12T10:15:00Z</clockIn>
    <clockOut>2020-01-12T11:15:00Z</clockOut>
    <declaredTips>5.50</declaredTips>
  </item>
</cards>
<start>
  <day>12</day>
  <month>1</month>
  <year>2020</year>
</start>
<end>
  <day>13</day>
  <month>1</month>
  <year>2020</year>
</end>

```

setTimecardsV3

This method takes in a concept id, store number, and a plethora of employee timecard information like wage rates, totals, start and end times to update time card information in Fourth for the given timeframe. ***This method does not work with Fourth Time & Attendance.**

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
breakMinutes	Int	false	Total minutes on break
breakType	String	false	Meal or Break
empPosId	Int	true	Employee POS ID
extId	Int	false	External employee ID
hsId	Int	false	Fourth Internal ID
jobExtId	Int	false	External Job ID
jobId	Int	true	Job Code POS ID
jobName	String	false	Job Code Name
ovtHrs	Float	false	Number of overtime hours worked

ovtMins	Float	false	Number of overtime minutes worked
ovtTtl	Float	false	Total amount of wages for overtime worked
ovtWage	Float	false	Overtime wage
regHrs	Float	true	Number of hours worked
regTtl	Float	true	Total amount of wages for regular hours worked
regWage	Float	true	Regular wage
spcHrs	Float	false	Number of special hours worked
spcTtl	Float	false	Total amount of wages for special hours worked
storeNum	Int	true	Store ID
businessDate	Array	false	Business date of timecards. • Day (int) • Month (Int) • Year (int)
clockIn	String	true	Date and time of clock in. (ex: 2020-0110T10:15:00Z)
clockOut	String	true	Date and time of clock out. (ex: 2020-0110T10:15:00Z)
start	Array	true	Start date of time cards • Day (Int) • Month (Int) • Year (Int)
end	Array	true	End date of time cards • Day (Int) • Month (Int)
			• Year (Int)

Example setTimecardsV3 call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<cards>
  <item>
    <empPosId>1</empPosId>
    <jobId>1</jobId>
    <regHrs>1</regHrs>
    <regTtl>9</regTtl>
    <regWage>9</regWage>
    <storeNum>280</storeNum>
    <businessDate>
      <day>13</day>
    
```

```

        <month>1</month>
        <year>2020</year>
    </businessDate>
    <clockIn>2020-01-13T09:00:00Z</clockIn>
    <clockOut>2020-01-13T10:00:00Z</clockOut>
</item>
</cards>
<start>
    <day>13</day>
    <month>1</month>
    <year>2020</year>
</start>
<end>
    <day>14</day>
    <month>1</month>
    <year>2020</year>
</end>

```

SetTimecardsV4

This method takes in a concept id, store number, and a plethora of employee timecard information like wage rates, totals, start and end times, breaks and tips to update time card information in Fourth for the given timeframe.

***This method is supported with Fourth Time & Attendance.**

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
empPosId	Int	true	Employee POS ID
extId	Int	true	Unique transaction ID for time card record
jobExtId	Int	true	External Job ID
clockIn	String	true	clock in timestamp for the shift in milliseconds since midnight, January 1, 1970 UTC
clockOut	String	true	clock out timestamp for the shift in milliseconds since midnight, January 1, 1970 UTC
Break	Array	true	
breakStart	Long	false	timestamp for the start of the break in

			milliseconds since midnight, January 1, 1970 UTC
breakEnd	Long	false	timestamp for the end of the break in milliseconds since midnight, January 1, 1970 UTC
breakType	Long	false	Supported Break Types are: "BREAK", "MEAL"
cashTips	Decimal	True	contains any cash tips earned
ccTips	Decimal	True	contains and credit card tips earned
serviceCharge	Decimal	True	contains any service charge amounts
tipPool	Decimal	True	the amount contributed to the tip pool

Example setTimecardsV4 call:

```

<concept>1</concept>
  <storeNum>13</storeNum>
  <item>
    <empPosId>2</empPosId>
    <extId>2363885598661</extId>
    <jobExtId>4</jobExtId>
    <clockIn>1623675600000</clockIn>
    <clockOut>1623706200000</clockOut>
    <breaks>
      <break>
        <breakEnd>1623683400000</breakEnd>
        <breakStart>1623682800000</breakStart>
        <breakType>BREAK</breakType>
      </break>
      <break>
        <breakEnd>1623691800000</breakEnd>
        <breakStart>1623690000000</breakStart>
        <breakType>MEAL</breakType>
      </break>
      <break>
        <breakEnd>1623697800000</breakEnd>
        <breakStart>1623697200000</breakStart>
        <breakType>BREAK</breakType>
      </break>
    </breaks>
    <cashTips>0.0</cashTips>
    <ccTips>0.0</ccTips>
    <serviceCharge>0.0</serviceCharge>
    <tipPool>0.0</tipPool>
  </item>

```

```

<item>
  <empPosId>5</empPosId>
  <extId>2363885598666</extId>
  <jobExtId>2</jobExtId>
  <clockIn>1623765600000</clockIn>
  <clockOut>1623787200000</clockOut>
  <breaks>
    <break>
      <breakEnd>1623781860000</breakEnd>
      <breakStart>1623780000000</breakStart>
      <breakType>MEAL</breakType>
    </break>
  </breaks>
  <cashTips>0.0</cashTips>
  <ccTips>0.0</ccTips>
  <serviceCharge>0.0</serviceCharge>
  <tipPool>0.0</tipPool>
</item>

```

Unsuccessful requests

Unsuccessful requests receive responses with HTTP status code 500 and a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client not found for concept: 123 store: 8033 </faultstring>
      <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/TimeCardService"/>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element.

faultstring	Description
client not found for concept: <concept number> store: <store number>	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.

client does not have permission to use the TimeCardService operation: <store name>: <store ID>	Insufficient permissions to use the API. Please contact Fourth to allow access.
Error with pos time card list <error details>	An unexpected error occurred when processing the request. Please ensure that the params are correct and resubmit the request.

Partially successful responses

The API will attempt to process all the timecards that pass validation. Invalid timecards will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of timecards that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setTimeCardsV4Response
xmlns:ns1="http://services.hotschedules.com/api/services/TimeCardService">
      <return>
        <errors> Breaks must have a BreakType, timecard will not be synced ;; {timecard
details} </errors>
        <errors> Breaks must have a valid breakId, timecard will not be synced ;; {timecard
details} </errors>
        <failCount>2</failCount>
        <success>true</success>
        <successCount>10</successCount>
      </return>
    </ns1:setTimeCardsV4Response>
  </soap:Body>
</soap:Envelope>
```

The table below lists possible validation errors. Note that each error is accompanied with timecard details to allow identifying the invalid record.

Validation error	Description
EmpPosId is invalid, timecard will not be synced	Employee POS ID of the timecard record is missing or invalid. A valid POS ID must be supplied to map the timecard to an employee.
Store has no employee with the provided EmpPosId, timecard will not be synced	No employee is found for the provided POS ID.
Job Identifier is invalid, timecard will not be synced	JobExtID of the timecard record is missing or invalid.
Store has no job associated with provided jobExtId, timecard will not be synced	No jobs are configured for the provided JobExtID.

Breaks must have a BreakType, timecard will not be synced	BreakType element is missing from a Break.
Breaks must provide a valid breakStart, timecard will not be synced	BreakStart element is missing from a Break.
Breaks must provide a valid breakEnd, timecard will not be synced	BreakEnd element is missing from a Break.
Breaks must have the BreakStart before BreakEnd, timecard will not be synced	Breaks that end before they start are invalid.
no valid time cards in POS list	The request either contains no timecards or all the timecards failed validation. In the latter case validation errors are provided in the response.

SalesService Available Methods

getRVCs

This method takes in a concept ID and store ID and returns the revenue centers for the specified location.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID

Response:

Name	Type	Description/Example
extId	Int	Revenue Center ID
groupLevel	Boolean	True = group level False = store level
revenueCenterName	String	Name of Revenue Center

Example getRVCs call:

```
<item>
  <extId>1</extId>
  <groupLevel>true</groupLevel>
  <revenueCenterName>To Go</revenueCenterName>
</item>
```

getSalesCats

This method takes in a concept ID and store ID and returns the sales categories for the specified location.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID

Response:

Name	Type	Description/Example
extId	Int	Sales Category ID
groupLevel	Boolean	Is sales category found at group level or not?
salesCategoryName	String	Name of Sales Category

Example getSalesCats call:

```
<item>
  <extId>1</extId>
  <groupLevel>true</groupLevel>
  <salesCategoryName>All</salesCategoryName>
</item>
```

getSalesItemsV3

This method takes in a concept ID, store ID, start and end dates, then returns the sales details for the specified location within the given range.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	Array	True	Start date of requested interval - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	True	End date of requested interval - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
clientId	Int	Unique Identifier for client provided by Fourth
empId	Int	POS ID of employee that did the transaction
extId	Int	External ID
rvc	Int	Revenue Center ID of the transaction
rvcName	String	Revenue Center Name of the transaction
salesCat	Int	Sales Category ID of the transaction

storeNum	Int	Store ID
ttl	Float	Total amount of transaction
businessDate	String	Business date of volume count
transDate	Array	Business date of transaction
transTime	Array	Time of transaction

Example getSalesItemsV3 call:

```

<sal:getSalesItemsV3>
  <concept>1</concept>
  <storeNum>2</storeNum>
  <start>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </start>
  <end>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </end>
</sal:getSalesItemsV3>

```

setSalesItems

This method takes in a concept ID, store ID, a start and end date and an array of WSSalesItem objects. The array contains sales items for a range of dates, corresponding to the start and end dates.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
catName	String	false	Sales Category Name
clientId	Int	true	Unique Identifier for client provided by Fourth
empId	Int	true	Employee POS ID
extId	Int	false	External Employee ID
rvc	Int	true	Revenue Center ID
rvcName	String	false	Revenue Center Name
salesCat	Int	true	Sales Category ID
storeNum	Int	true	Store ID

ttl	Float	true	Total of sale
businessDate	String	true	Business Date of sales items (ex: 2015-0916T00:00:00-05:00)
dateTime	String	true	Date and time of sale. (ex: 2015-0916T11:15:00-05:00)
start	String	true	Start date (ex: 2015-09-16T00:00:00-05:00)
end	String	true	End date (ex: 2015-09-17T00:00:00-05:00)

Example setSalesItems call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<sales>
  <item>
    <catName>Food</catName>
    <clientId>20390791</clientId>
    <empId>1</empId>
    <rv<1</rv<
    <rvName>Food</rvName>
    <salesCat>1</salesCat>
    <storeNum>1100030</storeNum>
    <ttl>20</ttl>
    <businessDate>2020-01-09T00:00:00Z</businessDate>
    <dateTime>2020-01-09T11:00:00Z</dateTime>
  </item>
</sales>
<start>2020-01-09T00:00:00Z</start>
<end>2020-01-10T00:00:00Z</end>

```

setSalesItemsV3

This method takes in a concept ID, store ID, a start and end date and an array of WSSalesItem objects. The array contains sales items for a range of dates, corresponding to the start and end dates.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
catName	String	false	Sales Category Name

clientId	Int	true	Unique Identifier for client provided by Fourth
empId	Int	true	Employee POS ID
extId	Int	false	External Employee ID
rvc	Int	true	Revenue Center ID
rvcName	String	false	Revenue Center Name
salesCat	Int	true	Sales Category ID
storeNum	Int	true	Store ID
ttl	Float	true	Total amount of transaction
businessDate	Array	false	• Day (Int) • Month (Int) • Year (Int)
transDate	Array	false	• Day (Int) • Month (Int) • Year (Int)
transTime	Array	false	• amPm (String) • hours (Int) • militaryTime (Boolean) • Minutes (Int) • Seconds (Int)
start	Array	true	• Day (Int) • Month (Int) • Year (Int)
end	Array	true	• Day (Int) • Month (Int) • Year (Int)

Example setSalesItemsV3 call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<sales>
  <item>
    <clientId>20390791</clientId>          <empId>1</empId>
    <rvc>1</rvc>
    <salesCat>1</salesCat>
    <storeNum>1100030</storeNum>
    <ttl>20</ttl>
    <businessDate>
      <day>10</day>
      <month>1</month>
      <year>2020</year>
    </businessDate>
  </item>
</sales>

```

```

    <transDate>
      <day>10</day>
      <month>1</month>
      <year>2020</year>
    </transDate>
    <transTime>
      <amPm>am</amPm>
      <hours>9</hours>
      <militaryTime>false</militaryTime>
      <minutes>0</minutes>
      <seconds>0</seconds>
    </transTime>
  </item>
</sales>
<start>
  <day>10</day>
  <month>1</month>
  <year>2020</year>
</start>
<end>
  <day>11</day>
  <month>1</month>
  <year>2020</year>
</end>

```

setSalesItemsV4

This method takes in a concept ID, store ID, and an array of Sales Item objects to update sales within a given time frame. Sending data for previous days will overwrite data that is already in the system.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
catName	String	false	Sales Category Name
clientId	Int	True (required?)	Unique Identifier for client provided by Fourth
empId	Int	true	Employee POS ID
extId	Int	false	External Employee ID
rvc	Int	true	Revenue Center ID
rvcName	String	false	Revenue Center Name

salesCat	Int	true	Sales Category ID
storeNum	Int	true	Store ID
ttl	Float	true	Total amount of transaction
businessDate	Array	false	• Day (Int) • Month (Int) • Year (Int)
transDate	Array	false	• Day (Int) • Month (Int) • Year (Int)
transTime	Array	false	• amPm (String) • hours (Int) • militaryTime (Boolean) • Minutes (Int) • Seconds (Int)
start	Array	true	• Day (Int) • Month (Int) • Year (Int)
end	Array	true	• Day (Int) • Month (Int) • Year (Int)

Example setSalesItemsV4 call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<sales>
  <item>
    <clientId>20390791</clientId>          <empId>1</empId>
    <rv<1</rv<
    <salesCat>1</salesCat>
    <storeNum>1100030</storeNum>
    <ttl>20</ttl>
    <businessDate>
      <day>11</day>
      <month>1</month>
      <year>2020</year>
    </businessDate>
    <transDate>
      <day>11</day>
      <month>1</month>
      <year>2020</year>
    </transDate>
    <transTime>

```



```

        <amPm>am</amPm>
        <hours>9</hours>
        <militaryTime>>false</militaryTime>
        <minutes>0</minutes>
        <seconds>0</seconds>
    </transTime>
</item>
</sales>
<start>
    <day>11</day>
    <month>1</month>
    <year>2020</year>
</start>
<end>
    <day>12</day>
    <month>1</month>
    <year>2020</year>
</end>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client does not have permission to use the SalesItemService operation
    </faultstring>
    <detail>
      <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/SalesItemService"/>
    </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element:

faultstring	Description
client does not have permission to use the SalesItemService operation: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.

Partially successful responses

The API will attempt to process all the sales items information that pass validation. Invalid sales items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of <sales> items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setSalesItemsV4Response xmlns:ns1="http://services.hotschedules.com/api/services/SalesItemService ">
      <return>
        <errors> POS Sales item list is null </errors>
        <failCount>1</failCount>
        <success>false</success>
        <successCount>0</successCount>
      </return>
    </ns1: setSalesItemsV4Response>
  </soap:Body>
</soap:Envelope>
```

Validation error	Description
POS Sales item list is null	Validate your sales items in request, empty request. Please ensure that the params are correct and resubmit the request.
Error comparing sales items: {}	Validate your sales items in request. Please ensure that the params are correct and resubmit the request.
error adding items for day {}	Validate your sales items in request. Please ensure that the params are correct and resubmit the request. Contact Fourth support if problems persist

ProjectedSalesService Available Methods

getProjectedSales

This method takes in a concept, store number, start and end dates then returns each business date and the projected sales totals for the entire day and each day part.

Call:

Name	Type	Required	Description/Example
------	------	----------	---------------------

concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	String	True	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
end	String	true	End date and time of requested range (ex: 2015-10-10T00:00:00-05:00)

Response:

Name	Type	Description/Example
businessDate	String	Business date within requested range.
dateTotal	Float	Total amount of projected sales during the business day.
dayPartTotals	Array	- dayPartEndTime (String) ◦ End time of day part. - dayPartName (String) - Name of date part - dayPartStartTime (String) - Start time of day part - dayPartTotal (Float) - Total amount of projected sales during the day part

Example getProjectedSales response:

```

<item>
  <businessDate>2019-10-20T00:00:00-05:00</businessDate>
  <dateTotal>8235.0</dateTotal>
  <dayPartTotals>
    <dayPartEndTime>1970-01-01T10:00:00-06:00</dayPartEndTime>
    <dayPartName>AM</dayPartName>
    <dayPartStartTime>1969-12-31T22:00:00-06:00</dayPartStartTime>
    <dayPartTotal>2977.0</dayPartTotal>
  </dayPartTotals>
  <dayPartTotals>
    <dayPartEndTime>1969-12-31T22:00:00-06:00</dayPartEndTime>
    <dayPartName>PM</dayPartName>
    <dayPartStartTime>1970-01-01T10:00:00-06:00</dayPartStartTime>
    <dayPartTotal>5258.0</dayPartTotal>
  </dayPartTotals>
</item>

```

getProjectedSalesV3

This method takes in a concept, store number, start and end dates then returns each business date and the projected sales totals for the entire day and each day part.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	Array	True	Start date of requested range - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	End date of requested range - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
businessDate	Array	Business date within requested range: - Day (Int) - Month (Int) - Year (Int)
dateTotal	Float	Total amount of projected sales during the business day.
dayPartTotals	Array	- dayPartEndTime (Array) - amPm - Hours - militaryTime - Minutes - seconds - dayPartName (String) - dayPartStartTime (Array) - amPm - Hours - militaryTime - Minutes - seconds - dayPartTotal (Float) - Total amount of projected sales during the day part

Example getProjectedSalesV3 response:

<item>

```

    <businessDate>
      <day>18</day>
      <month>10</month>
      <year>2019</year>
    </businessDate>
    <dateTotal>8572.0</dateTotal>
    <dayPartTotals>
      <dayPartEndTime>
        <amPm/>
        <hours>16</hours>
        <militaryTime>true</militaryTime>
        <minutes>0</minutes>
<seconds>0</seconds>
      </dayPartEndTime>
      <dayPartName>AM</dayPartName>
      <dayPartStartTime>
        <amPm/>
        <hours>4</hours>
        <militaryTime>true</militaryTime>
        <minutes>0</minutes>
<seconds>0</seconds>
      </dayPartStartTime>
      <dayPartTotal>3069.0</dayPartTotal>
    </dayPartTotals>
    <dayPartTotals>
      <dayPartEndTime>
        <amPm/>
        <hours>4</hours>
        <militaryTime>true</militaryTime>
        <minutes>0</minutes>
<seconds>0</seconds>
      </dayPartEndTime>
      <dayPartName>PM</dayPartName>
      <dayPartStartTime>
        <amPm/>
        <hours>16</hours>
        <militaryTime>true</militaryTime>
        <minutes>0</minutes>
<seconds>0</seconds>
      </dayPartStartTime>
      <dayPartTotal>5503.0</dayPartTotal>
    </dayPartTotals>
  </item>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> client not found for concept: 123 store: 8033 </faultstring>
      <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/ProjectedSalesService"/>
        </detail>
      </soap:Fault>
    </soap:Body>
  </soap:Envelope>
```

The following errors can be returned in the faultstring element.

faultstring	Description
client does not have permission to use the ProjectedSalesService operation: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.

LaborService Available Methods

getLaborByBusDate

This method takes in a concept, store number, business start and end date and a labor type, then returns the volume of the labor type requested within the given interval.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
busStart	Array	True	- Day (Int) - Month (Int) - Year (Int)
busEnd	Array	true	- Day (Int) - Month (Int) - Year (Int)
laborType	String	true	Type of labor requested. Allowed types are:

			• optimal • forecasted • scheduled
--	--	--	--

Response:

Name	Type	Description/Example
intervalDate	Array	• Day (Int) • Month (Int) • Year (Int)
intervalTime	Array	• Hours (Int) • militaryTime (Boolean) • Minutes (Int) • Seconds (Int)
laborType	String	Labor type requested.
volume	Float	Amount of volume during the interval

Example getLaborByBusDate response:

```

<interval>
  <intervalDate>
    <day>10</day>
    <month>1</month>
    <year>2020</year>
  </intervalDate>
  <intervalTime>
    <hours>4</hours>
    <militaryTime>true</militaryTime>
    <minutes>0</minutes>
    <seconds>0</seconds>
  </intervalTime>
  <laborType>scheduled</laborType>
  <volume>0.0</volume>
</interval>
<interval>
  <intervalDate>
    <day>10</day>
    <month>1</month>
    <year>2020</year>
  </intervalDate>
  <intervalTime>
    <hours>4</hours>
    <militaryTime>true</militaryTime>

```

```

    <minutes>30</minutes>
    <seconds>0</seconds>
  </intervalTime>
  <laborType>scheduled</laborType>
  <volume>0.0</volume>
</interval>

```

getLaborByJobAndInterval

This method takes in a concept, store number, business start and end date and a labor type, then returns the volume of the labor type requested within the given interval.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	Array	True	- Day (Int) - Month (Int) - Year (Int)
end	Array	true	- Day (Int) - Month (Int) - Year (Int)
laborType	String	true	Type of labor requested. Allowed types are: - optimal - forecasted - scheduled

Response:

Name	Type	Description/Example
intervalDate	Array	- Day (Int) - Month (Int) - Year (Int)
intervalTime	Array	- Hours (Int) - militaryTime (Boolean) - Minutes (Int) - Seconds (Int)
laborType	String	Labor type requested.
volume	Float	Amount of volume during the interval

Example getLaborByJobAndInterval response:


```

<intervalDate>
  <day>10</day>
  <month>1</month>
  <year>2020</year>
</intervalDate>
<intervalTime>
  <hours>0</hours>
  <militaryTime>>true</militaryTime>
  <seconds>0</seconds>
  <minutes>0</minutes>
</intervalTime>
<laborType>scheduled</laborType>
<volume>0.0</volume>
</interval>
<interval>
  <intervalDate>
    <day>10</day>
    <month>1</month>
    <year>2020</year>
  </intervalDate>
  <intervalTime>
    <hours>0</hours>
    <militaryTime>>true</militaryTime>
    <minutes>30</minutes>
    <seconds>0</seconds>
  </intervalTime>
  <laborType>scheduled</laborType>
  <volume>0.0</volume>
</interval>

```

getLaborByJobAndIntervalV2

This method takes in a concept, store number, business start and end date and a labor type, then returns the volume of the labor type requested within the given interval.

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>

```

```

    <faultcode>soap:Server</faultcode>
    <faultstring> client does not have permission to use the labor service: {} </faultstring>
    <detail>
      <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/LaborService"/>
    </detail>
  </soap:Fault>
</soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element.

faultstring	Description
client does not have permission to use the labor service: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.
client does not have permission to use the getLaborByJobAndInterval operation: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.
client not found for concept: {} store: {}	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
Failed to parse labor type	Labor type is not present in request. Please ensure that the params are correct and resubmit the request.

getLaborByPositionAndInterval

This method will take a concept ID, store number, start and end dates and a labor type and return a list of total labor by position for each interval in the date range requested for that concept, store and labor type. Intervals are configured during initial setup for the customer and are typically 30 minutes or 15 minutes.

Input (Literal)

The inputs of this method are the arguments defined by the following table.

Argument	Type	Occurs	Description
All		1..1	
concept	int	1..1	The identifier for the location's concept. Must be unique within the company. Contact HotSchedules if you're not sure about this value

storeNum	String	1..1	Numeric or alphanumeric identifier configured for the store. Must be unique within the concept.
start	hsSimpleDate	1..1	Start date for the range of data requested
end	hsSimpleDate	1..1	End date for the range of data requested
laborType	laborType	1..1	Type of labor requested. Allowed types are "optimal", "forecasted" and "scheduled".

Output (Literal)

The outputs of this method are the arguments defined by the following table.

Argument	Type	Occurs	Description
All		1..1	

return	wsLaborPositionArray	1..1	Returns a wsLaborPositionArray object, which is an array of wsLaborPosition objects. Each wsLaborPosition object contains - a positionName (the name of the position) - a wsLaborInterval object, which can contain + an intervalDate object (an hsSimpleDate that determines the date for this particular interval) + an intervalTime object (an hsSimpleTime that determines the start time for this particular interval. Interval length is implied by the span between two intervalTime objects) + a laborType (an enum which can be "optimal", "forecasted", "scheduled") + a volume value (which represents the value for the laborDate type for this particular interval, determined by intervalDate and intervalTime) ** Note about labor types. "Scheduled" labor refers to shifts that have been scheduled and posted and are assigned to an employee. This will not include house shifts or shifts not assigned to an employee. "forecasted" labor refers to labor shifts generated after forecast labor drivers and applying labor shift generation rules within the Activity based
--------	----------------------	------	--

			forecasting module in HotSchedules "optimal" labor refers to labor shifts generated by using actual labor driver values and applying labor shift generation rules specified in the Activity based forecasting module in HotSchedules. It represents the labor shifts that would have been generated using an exact forecast of labor drivers
--	--	--	---

Faults

Name	Content	Description
ApiPermissionException		If client does not have a permission to use the LaborService.getLaborByPositionAndInterval operation. 3033 Client Permission

Example

Example getLaborByJobAndInterval response:

```
<?xml version="1.0" encoding="UTF8"?>
<soapenv:Envelope xmlns:lab="http://services.hotschedules.com/api/services/LaborService"
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header>    <wsse:Security
xmlns:wsse="http://docs.oasisopen.org/wss/2004/01/oasis200401wsswssecuritysecect1.0.xsd"
xmlns:wsu="http://docs.oasisopen.org/wss/2004/01/oasis200401wsswssecurityutility1.0.xsd" soapenv:mustUnderstand="1">
    <wsse:UsernameToken
wsu:Id="UsernameToken63A716DE011BE2D649145262747932814">
      <wsse:Username>Username</wsse:Username>
      <wsse:Password
Type="http://docs.oasisopen.org/wss/2004/01/oasis200401wssusnametokenprofile1.0#PasswordText"> Password
      </wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soapenv:Header>
  <soapenv:Body>
    <lab:getLaborByPositionAndInterval>
      <concept>101</concept>
      <storeNum>246</storeNum>
      <start>
        <day>2</day>
        <month>8</month>
      </start>
      <end>
        <day>3</day>
```

```

        <month>8</month>
<year>2022</year>      </end>
    <laborType>scheduled</laborType>
</lab:getLaborByPositionAndInterval>
</soapenv:Body> </soapenv:Envelope>

```

RESPONSE:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:getLaborByPositionAndIntervalResponse
xmlns:ns1="http://services.hotschedules.com/api/services/LaborService">
      <return xmlns:ns2="http://services.hotschedules.com/api/services/LaborService">
<item>
      <interval>
        <intervalDate>
          <day>2</day>
          <month>8</month>
<year>2022</year>
        </intervalDate>
        <intervalTime>
          <hours>0</hours>
          <militaryTime>true</militaryTime>
          <minutes>0</minutes>
          <seconds>0</seconds>
        </intervalTime>
        <laborType>scheduled</laborType>
        <volume>0.0</volume>
      </interval>
      <interval>
        <intervalDate>
          <day>2</day>
          <month>8</month>
<year>2022</year>
        </intervalDate>
        <intervalTime>
          <hours>0</hours>
          <militaryTime>true</militaryTime>
          <minutes>30</minutes>
          <seconds>0</seconds>
        </intervalTime>
        <laborType>scheduled</laborType>

```

```

        <volume>0.0</volume>
    </interval>
    <interval>
        <intervalDate>
            <day>2</day>
            <month>8</month>
<year>2022</year>
        </intervalDate>
        <intervalTime>
            <hours>1</hours>
            <militaryTime>true</militaryTime>
            <minutes>0</minutes>
            <seconds>0</seconds>
        </intervalTime>
        <laborType>scheduled</laborType>
        <volume>0.0</volume>
    </interval>
    <interval>
        <intervalDate>
            <day>2</day>
            <month>8</month>
<year>2022</year>
        </intervalDate>
        <intervalTime>
            <hours>1</hours>
            <militaryTime>true</militaryTime>
            <minutes>30</minutes>
            <seconds>0</seconds>
        </intervalTime>
        <laborType>scheduled</laborType>
        <volume>0.0</volume>
    </interval>
    <positionName>Bartender LP</positionName>
</item>
</return>
</ns1:getLaborByPositionAndIntervalResponse>
</soap:Body>
</soap:Envelope>

```


VolumeService Available Methods

getDriversByInterval

This method will take a concept ID, store number, start and end dates, volume type, and data type and return a list of total driver amount for each interval in the date range requested for that concept, store and labor type.

*Intervals are configured during initial setup for the customer and are typically 30 minutes or 15 minutes.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
startDate	Array	true	Start date of requested interval. • Day (Int) - 1-30 or 31 • Month (Int) - 1-12 • Year (Int) - YYYY, Ex: 2019
endDate	Array	true	End date of requested interval • Day (Int) - 1-30 or 31 • Month (Int) - 1-12 • Year (Int) - YYYY, Ex: 2019
volumeType	String	true	Classification of driver requested. Allowed types are: • SALES • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS
dataType	String	true	Type of driver requested. Allowed types are: • ACTUAL • ADJ_FORECASTED • PRE_ADJ_FORECASTED

Response:

Name	Type	Description/Example
companyExtRef	Int	Internal Company External Reference ID
conceptExtRef	Int	Internal Concept External Reference ID
driverAmount	Float	Quantity of driver for interval
driverClass	String	Classification of driver requested. Allowed types are: • SALES • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS
driverType	String	Type of driver requested. Type of driver requested. • ACTUAL • ADJ_FORECASTED • PRE_ADJ_FORECASTED
intervalEndDate	Array	End date of requested interval. • Day (Int) • Month (Int) • Year (Int)
intervalEndTime	Array	End time of requested interval. • hours (Int) • militaryTime (Boolean) • Minutes (Int) • Seconds (Int)
intervalStartDate	Array	Start date of requested interval • Day (Int)
		• Month (Int) • Year (Int)
intervalStartTime	Array	Start time of requested interval • hours (Int) • militaryTime (Boolean) • Minutes (Int) • Seconds (Int)
storeExtRef	Int	Store ID

Example getDriversByInterval Response:

```
<item>
  <companyExtRef>234542</companyExtRef>
  <conceptExtRef>1</conceptExtRef>
  <driverAmount>0.0</driverAmount>
  <driverClass>DELIVERIES</driverClass>
  <driverType>PRE_ADJ_FORECASTED</driverType>
  <intervalEndDate>
    <day>18</day>
    <month>10</month>
    <year>2019</year>
  </intervalEndDate>
  <intervalEndTime>
    <hours>0</hours>
    <militaryTime>true</militaryTime>
    <minutes>30</minutes>
    <seconds>0</seconds>
  </intervalEndTime>
<intervalStartDate>
  <day>18</day>
  <month>10</month>
  <year>2019</year>
</intervalStartDate>
<intervalStartTime>
  <hours>0</hours>
  <militaryTime>true</militaryTime>
  <minutes>0</minutes>
  <seconds>0</seconds>
</intervalStartTime>
  <storeExtRef>2</storeExtRef>
</item>
```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> Error in VolumeService </faultstring>
      <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/VolumeService"/>
```

```

</detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element:

faultstring	Description
client not found for concept: {} store: {}	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
client does not have permission to use the VolumeService operation	Insufficient permissions to use the API. Please contact Fourth to allow access.
client does not have permission to use the VolumeService getDriversByInterval() operation	Insufficient permissions to use the API. Please contact Fourth to allow access.
ERROR: invalid volumeType specified	Invalid volumeType. Please ensure that the params are correct and resubmit the request.
UNDEFINED volumeType parameter received, no volume data will be returned	Invalid volumeType. Please ensure that the params are correct and resubmit the request.
ERROR: invalid dataType specified	Invalid dataType. Please ensure that the params are correct and resubmit the request.
ERROR: start date cannot be null	Invalid date. Please ensure that the params are correct and resubmit the request.
ERROR: end date cannot be null	Invalid date. Please ensure that the params are correct and resubmit the request.
ERROR: start date cannot be after end date when calling getTemplates()	Start date is after the end date. Please ensure that the params are correct and resubmit the request.
ERROR: start date cannot be more than 10 years from now	Date limitation. Please ensure that the params are correct and resubmit the request.
ERROR: end date cannot be more than 10 years from now	Date limitation. Please ensure that the params are correct and resubmit the request.
ERROR: start and end dates may not be more than 30 days apart	Date range limitation. Please ensure that the params are correct and resubmit the request.
Error in VolumeService	Please ensure that the params are correct and resubmit the request. Please contact Fourth support if the problem persists

getGuestCounts

This method will take a concept ID, store number, start and end dates and return a list of guest counts for the date range requested.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
startDate	String	true	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
endDate	String	true	End date and time of requested range (ex: 2015-10-16T00:00:00-05:00)

Response:

Name	Type	Description/Example
businessDate	String	Business date of volume count
dateTime	String	Date and time of volume count
guestCount	Int	Volume amount
rvcExtId	Int	Revenue Center ID

Example getGuestCounts Response:

```
<item>
  <businessDate>2015-10-09T00:00:00-05:00</businessDate>
  <dateTime>2015-10-09T11:00:00-05:00</dateTime>
  <guestCount>4.0</guestCount>
  <rvcExtId>1</rvcExtId>
</item>
```

getVolumeCounts

This method will take a concept ID, store number, start and end dates and a volume type and return a list of volume counts for the date range requested. Supported Volume Types are: "TABLES", "ENTRÉE", "GUESTS", "DELIVERIES", "PRODUCTS", and "TRANSACTIONS".

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
volumeType	String	true	Classification of volume requested. Allowed types are: • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS
startDate	String	true	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
endDate	String	true	End date and time of requested range (ex: 2015-10-16T00:00:00-05:00)

Response:

Name	Type	Description/Example
businessDate	String	Business date of volume count
dateTime	String	Date and time of volume count
rvcExtId	Int	Revenue Center ID
volumeAmount	Int	Volume Amount
volumeType	String	Classification of volume requested. • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS

Example getVolumeCounts Response:

```
<item>                <businessDate>2015-10-09T00:00:00-05:00</businessDate>
```

```

<dateTime>2015-10-09T11:00:00-05:00</dateTime>
<rvcExtId>1</rvcExtId>
<volumeAmount>7.0</volumeAmount>
<volumeType>GUESTS</volumeType>
</item>

```

setForecastDriversV2

This method takes in a start date, end date, start time, end time, volume amount, volume type, and a revenue center for the purpose of submitting forecasted volume drivers to Fourth from a third-party system or point of sale. Using this method you need to send an entire business days' worth of intervals, matching what is configured within Fourth. *Data sent **must** fall within the location's day parts. If any interval sent falls outside of the intended date, data loss may occur.

Intervals are configured during the initial setup for the customer and are typically 30 minutes or 15 minutes.

Call:

Name	Type	Required	Description/Example
Concept	Int	true	Concept ID - Provided by Fourth.
StoreNum	String	true	Alphanumeric Store ID
workweekStartDate	Array	true	The work week start date. - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
driverAmount	Float	true	The amount of the requested driver.
intervalEndDate	Array	false	End date of requested interval - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
intervalEndTime	Array	false	End time of requested interval - militaryTime (Boolean) - true or false - amPm (String) - am or pm (only needed if militaryTime = false) - hours (Int) 0-12 if militaryTime = false

			○ 0-23 if militaryTime = true • Minutes (Int) - 0-59 • Seconds (Int) - 0-59
intervalStartDate	Array	false	Start date of requested interval • Day (Int) - 1-30 or 31 • Month (Int) - 1-12 • Year (Int) - YYYY, Ex: 2019
intervalStartTime	Array	false	Start time of requested interval • militaryTime (Boolean) - true or false • amPm (String) - am or pm (only needed if militaryTime = false) • hours (Int) ○ 0-12 if militaryTime = false ○ 0-23 if militaryTime = true • Minutes (Int) - 0-59 • Seconds (Int) - 0-59
rvcId	Int	true	Revenue Center ID
volumeType	String	false	Classification of volume requested. Allowed types are: • SALES • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS

Example setForecastDriversV2 Call:

```

<vol:setForecastDriversV2>
<concept>280</concept>
  <storeNum>abc1100030</storeNum>
  <workweekStartDate>
    <day>20</day>
    <month>10</month>
    <year>2019</year>
  </workweekStartDate>

```



```

<driverData>
  <item>
    <driverAmount>71</driverAmount>
    <intervalEndDate>
      <day>21</day>
      <month>2</month>
      <year>2020</year>
    </intervalEndDate>
    <intervalEndTime>
      <amPm>PM</amPm>
      <hours>13</hours>
      <militaryTime>true</militaryTime>
      <minutes>15</minutes>
      <seconds>0</seconds>
    </intervalEndTime>
    <intervalStartDate>
      <day>21</day>
      <month>2</month>
      <year>2020</year>
    </intervalStartDate>
    <intervalStartTime>
      <amPm>PM</amPm>
      <hours>13</hours>
      <militaryTime>true</militaryTime>
      <minutes>0</minutes>
    </intervalStartTime>
    <seconds>0</seconds>
    <rvId>0</rvId>
    <volumeType>Sales</volumeType>
  </item>
</driverData>
</vol:setForecastDriversV2>

```

setGuestCounts

This method takes in a concept ID, store ID, business date, date time, guest count (must be a positive integer) and a revenue center for the purpose of submitting actual guest count drivers to Fourth from a third party system or point of sale.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
businessDate	String	true	Business date of volume count (ex: 2015-10-09T00:00:00-05:00)
dateTime	String	true	Date and time of volume count (ex: 2015-10-09T11:15:00-05:00)
guestCount	Int	true	Volume Amount
rvcExtId	Int	true	Revenue Center ID

Example setGuestCounts Call:

```
<vol:setGuestCounts>
  <concept>1</concept>
  <storeNum>2</storeNum>
  <guests>
    <item>
      <businessDate>2015-10-09T00:00:00-05:00</businessDate>
      <dateTime>2015-10-09T11:15:00-05:00</dateTime>
      <guestCount>5</guestCount>
      <rvcExtId>1</rvcExtId>
    </item>
  </guests>
</vol:setGuestCounts>
```

setVolumeCounts

This method takes in a concept ID, store ID, business date, date time, volume amount, volume type, and a revenue center for the purpose of submitting actual volume drivers to Fourth from a third party system or point of sale.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
businessDate	String	true	Business date of volume count (ex: 2015-10-09T00:00:00-05:00)
dateTime	String	true	Date and time of volume count (ex: 2015-10-09T11:15:00-05:00)
rvcExtId	Int	true	Revenue Center ID
volumeAmount	Int	true	Volume Amount
volumeType	String	true	Classification of volume requested. Allowed types are: • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS

Example setVolumeCounts Call:

```
<vol:setVolumeCounts>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
  <volumeData>
    <item>
      <businessDate>2015-10-09T00:00:00-05:00</businessDate>
      <dateTime>2015-10-09T00:00:00-05:00</dateTime>
      <rvcExtId>1</rvcExtId>
      <volumeAmount>5</volumeAmount>
      <volumeType>GUESTS</volumeType>
    </item>
  </volumeData>
</vol:setVolumeCounts>
```

setVolumeCountV2

This method takes in a concept ID, store ID, business date, date time, volume amount, volume type, and a revenue center for the purpose of submitting actual volume drivers to Fourth from a third-party system or point of sale. Using the authentication from the username token and the concept and store IDs, the server will resolve which Fourth client this sync is for. The array contains volume driver counts for a range of dates, corresponding to the start and end dates. The server-side logic can handle overlapping data (i.e. if you sync 7 day's worth of timecards, every day, 6 days of it will be "overlapping" data) and will insert and update data as needed. If the guest are already in the HS database and do not need to be updated, then nothing will change. This method returns a WSReturn object.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID

Volume Array

Name	Type	Required	Description/Example
businessDate	String	true	Business Date of transaction
dateTime	String	true	Date and time of volume count
rvcExtId	Int	true	Revenue Center ID
volumeAmount	Int	true	Value of the volume count for the transaction
volumeType	String	true	Classification of driver requested. • Guests • Tables • Entrees • Deliveries • Transactions • Products

Example setVolumeCountV2 call:

```
<concept>280</concept>
<storeNum>1100030</storeNum>
<volumeData>
  <item>
    <businessDate>2015-10-09T00:00:00-05:00</businessDate>
```

```

    <dateTime>2015-10-09T00:00:00-05:00</dateTime>
    <rvcExtId>1</rvcExtId>
    <volumeAmount>5</volumeAmount>
    <volumeType>GUESTS</volumeType>
  </item>
</volumeData>

```

setVolumeCountsV3

This method takes in a concept ID, store ID, business date, date time, volume amount, volume type, and a revenue center for the purpose of submitting actual volume drivers to Fourth from a third party system or point of sale.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
businessDate	String	true	Business date of volume count (ex: 2015-10-09T00:00:00-05:00)
dateTime	String	true	Date and time of volume count (ex: 2015-10-09T11:15:00-05:00)
rvcExtId	Int	true	Revenue Center ID
volumeAmount	Int	true	Volume Amount
volumeTypeName	String	true	Classification of volume requested. Allowed types are: • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS

Example setVolumeCountsV3 Call:

```
<vol:setVolumeCountsV3>
```

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<volumeData>
  <item>
    <businessDate>2015-10-09T00:00:00-05:00</businessDate>
    <dateTime>2015-10-09T00:00:00-05:00</dateTime>
    <rvcExtId>1</rvcExtId>
    <volumeAmount>5</volumeAmount>
    <volumeType>
      <volumeTypeName>GUESTS</volumeTypeName>
    </volumeType>
  </item>
</volumeData>
</vol:setVolumeCountsV3>

```

setVolumeCountsV4

This method takes in a concept ID, store ID, business date, date time, volume amount, volume type, and a revenue center for the purpose of submitting actual volume drivers to Fourth from a third party system or point of sale.

Call:

Name	Type	Required	Description/Example
concept	Int	true	Concept ID - Provided by Fourth.
storeNum	Int	true	Store ID
businessDate	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
rvcExtId	Int	true	Revenue Center ID
salesDate	Array	true	- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

salesTime	Array	true	• militaryTime (Boolean) - true or false • amPm (String) - am or pm (only needed if militaryTime = false) • hours (Int) ◦ 0-12 if militaryTime = false ◦ 0-23 if militaryTime = true • Minutes (Int) - 0-59 • Seconds (Int) - 0-59
volumeAmount	Int	true	Volume Amount
volumeTypeName	String	true	Classification of volume requested. Allowed types are: • GUESTS • TABLES • ENTREES • DELIVERIES • TRANSACTIONS • PRODUCTS

Example setVolumeCountsV4 Call:

```

<vol:setVolumeCountsV4>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
  <volumeData>
    <item>
      <businessDate>
        <day>10</day>
        <month>10</month>
        <year>2019</year>
      </businessDate>
      <rvcExtId>1</rvcExtId>
      <salesDate>
        <day>10</day>
        <month>10</month>
        <year>2019</year>
      </salesDate>
      <salesTime>
        <amPm>am</amPm>
        <hours>11</hours>
        <militaryTime>true</militaryTime>
      </salesTime>
    </item>
  </volumeData>
</vol:setVolumeCountsV4>

```

```

        <minutes>15</minutes>
        <seconds>0</seconds>
    </salesTime>
    <volumeAmount>5</volumeAmount>
    <volumeType>
        <volumeTypeName>GUESTS</volumeTypeName>
    </volumeType>
</item>
</volumeData>
</vol:setVolumeCountsV4>

```

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> Error in VolumeService </faultstring>
      <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/VolumeService"/>
        </detail>
      </soap:Fault>
    </soap:Body>
  </soap:Envelope>

```

Partially successful responses

The API will attempt to process all the volume information that pass validation. Invalid volume data items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of <volumeData> items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setVolumeCountsV4Response
xmlns:ns1="http://services.hotschedules.com/api/services/VolumeService">
      <return>
        <errors>ERROR: setVolume() invalid parameters</errors>
        <failCount>1</failCount>
        <success>>false</success>
        <successCount>0</successCount>
      </return>
    </ns1:setVolumeCountsV4Response>
  </soap:Body>
</soap:Envelope>

```



```

</ns1:setVolumeCountsV4Response>
</soap:Body>
</soap:Envelope>

```

Validation error	Description
ERROR: volume data list is empty, please send actual data with this call. Bailing out.	Empty volume data in request. Please ensure that the params are correct and resubmit the request.
Error: Invalid volume type specified ...	Invalid volume type. Please ensure that the params are correct and resubmit the request.
Error: Invalid RVC ID, ...	Invalid RVC ID. Please ensure that the params are correct and resubmit the request.
Error: Invalid Volume Data Amount, must be > 0, ...	Volume amount invalid. Please ensure that the params are correct and resubmit the request.
ERROR: setVolume() invalid parameters	Most likely invalid volumeType provided. Please ensure that the params are correct and resubmit the request.

TimeOffService Available Methods

getApprovedPTO

This method takes in a concept, store number, start and end dates then returns any approved PTO within the specified time frame and store.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	Array	True	Start date of requested range - Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019
end	Array	true	End date of requested range
			- Day (Int) - 1-30 or 31 - Month (Int) - 1-12 - Year (Int) - YYYY, Ex: 2019

Response:

Name	Type	Description/Example
clientId	Int	Fourth internal Store ID.
employeeFirstName	String	Employee's first name
employeeHrId	Int	Employees HR ID. This ID is unique to the employee across the company.
employeeLastName	String	Employee's last name
employeePosId	Int	Employee's POS ID
end	String	End date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
jobId	Int	Job Code POS ID
jobName	String	Job Code name
jobRate	Float	Pay rate for job code
start	String	Start date and time of requested range (ex: 2015-10-09T00:00:00-05:00)
timeOffHours	Float	Amount of PTO in hours
timeOffTypeCode	Int	ID of time off

Example getApprovedPTO response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:getApprovedPTOResponse
xmlns:ns1="http://services.Fourth.com/api/services/TimeOffService">
      <return>
        <item>
          <clientId>2</clientId>
          <employeeFirstName>Brianna</employeeFirstName>
          <employeeHrId>1214</employeeHrId>
          <employeeLastName>Thomas</employeeLastName>
          <employeePosId>1046</employeePosId>
          <end>2020-07-10T00:00:00.000-05:00</end>
          <jobId>1</jobId>
          <jobName>Cook</jobName>
          <jobRate>1.0</jobRate>
          <start>2020-07-09T00:00:00.000-05:00</start>
          <timeOffHours>8.00</timeOffHours>
          <timeOffTypeCode>11</timeOffTypeCode>
        </item>
      </return>
    </ns1:getApprovedPTOResponse>
  </soap:Body>
```

</soap:Envelope>

CertificationService Available Methods

getClientCertifications

This method takes in a concept ID and a store ID and returns client certification objects.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID

Response:

Name	Type	Description/Example
certExtRef	Int	Certification ID
certName	String	Certification Name
certType	Int	Certification Type ID
certTypeDescription	String	Certification Description
groupLevel	Boolean	True = group level False = store level

Example getClientCertifications response:

```
<return>
  <item>
    <certExtRef>-1</certExtRef>
    <certName>Employee 6 Month Review</certName>
    <certType>2</certType>
    <certTypeDescription>Expires Without AutoInactivation</certTypeDescription>
    <groupLevel>>false</groupLevel>
  </item>
</return>
```

getEmployeeCertifications

This method takes in a concept ID, store ID and employee POS ID and returns an array of employee certification objects.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
empPosid	Int	True	Employee POS ID

Response:

Name	Type	Description/Example
certExtRef	Int	Certification ID
certName	String	Certification Name
certType	Int	Certification Type ID
certTypeDescription	String	Certification Description
groupLevel	Boolean	True = group level False = store level
empPosId	Int	Employee's POS ID
expirationDate	String	Date when certification expires

Example getEmployeeCertification response:

```
<item>
  <certExtRef>-1</certExtRef>
  <certName>Alcohol Server Training</certName>
  <certType>2</certType>
  <certTypeDescription>Expires Without AutoInactivation</certTypeDescription>
  <groupLevel>>false</groupLevel>
  <empPosId>5</empPosId>
  <expirationDate>2019-11-21T00:00:00-06:00</expirationDate>
</item>
```

removeEmployeeCertifications

This method takes in a concept, store number, certification information and an employee ID, then will remove the specified certification from the employee.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
certExtRef	Int	True	Certification ID
certName	String	false	Certification Name
certType	String	true	Certification Type ID
certTypeDescription	String	false	Certification Description
groupLevel	Boolean	true	True = group level False = store level
empPosId	Int	true	Employee's POS ID
expirationDate	String	false	Date when certification expires

Example removeEmployeeCertification response:

```

<cer:removeEmployeeCertifications>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
  <certs>
    <item>
      <certExtRef>1</certExtRef>
      <certName>Alcohol Server Training</certName>
      <certType>2</certType>
      <certTypeDescription>Expires Without AutoInactivation</certTypeDescription>
      <groupLevel>false</groupLevel>
      <empPosId>94</empPosId>
      <expirationDate>2019-11-21T00:00:00-06:00</expirationDate>
    </item>
  </certs>
</cer:removeEmployeeCertifications>

```

setEmployeeCertifications

This method takes in a concept, store number, certification information and an employee POS ID to assign the specified certification to the employee.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
certExtRef	Int	True	Certification ID

certName	String	False	Certification Name
certType	Int	True	Certification Type ID
certTypeDescription	String	False	Certification Description
groupLevel	Boolean	True	True = group level False = store level
empPosId	Int	True	Employee's POS ID
expirationDate	String	False	Date when certification expires

Example setEmployeeCertifications Call:

```

<cer:setEmployeeCertifications>
  <concept>280</concept>
  <storeNum>1100030</storeNum>
  <certs>
    <!--Zero or more repetitions:-->
    <item>
      <certExtRef>1</certExtRef>
      <!--Optional:-->
      <certName>Alcohol Server Training</certName>
      <certType>2</certType>
      <!--Optional:-->
      <certTypeDescription>Expires Without AutoInactivation</certTypeDescription>
      <groupLevel>>false</groupLevel>
      <empPosId>94</empPosId>
      <!--Optional:-->
      <expirationDate>2019-11-21T00:00:00-06:00</expirationDate>
    </item>
  </certs>
</cer:setEmployeeCertifications>

```

setEmployeeCertificationsV2

This method takes in a concept, store number, certification information and an employee POS ID to assign the specified certification to the employee.

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>

```

```

    <faultcode>soap:Server</faultcode>
    <faultstring> client does not have permission to use the CertificationService operation: {}
</faultstring>
    <detail>
        <ns1:Exception
xmlns:ns1="http://services.hotschedules.com/api/services/CertificationService"/>
    </detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>

```

The following errors can be returned in the faultstring element.

faultstring	Description
client does not have permission to use the CertificationService: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.
client not found for concept: {} store: {}	The API was unable to locate the store by concept and store number. Please ensure that the params are correct and resubmit the request.
client does not have permission to use the CertificationService removeEmployeeCertifications or setEmployeeCertifications operations: {}	Insufficient permissions to use the API. Please contact Fourth to allow access.

Partially successful responses

The API will attempt to process all the certs items that pass validation. Invalid certs items will be skipped, and an appropriate error message will be added in the response. Additionally, successCount and failCount will present the number of certs items that passed validation vs were invalid. The response HTTP status code is 200 regardless of whether the request contained invalid records.

Here is an example of a response with validation errors:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ns1:setEmployeeCertificationV2Response
xmlns:ns1="http://services.hotschedules.com/api/services/ CertificationService ">
      <return>
        <errors> ERROR: -1 is not a correct external reference value to identify a certification
.</errors>
        <failCount>1</failCount>
        <success>>false</success>
        <successCount>0</successCount>

```

```

    </return>
  </ns1: setEmployeeCertificationV2Response>
</soap:Body>
</soap:Envelope>

```

Validation error	Description
ERROR: -1 is not a correct external reference value to identify a certification.	Please ensure that the params are correct and resubmit the request.
ERROR: invalid emp posId, this cert will be skipped. clientId, posId {}	Please ensure that the params are correct and resubmit the request.
ERROR: invalid certExtRef, this cert will be skipped: {}	Please ensure that the params are correct and resubmit the request.
ERROR: invalid cert expiration date, this cert will be skipped. certExtRef, expiration date {}	Please ensure that the params are correct and resubmit the request.
Invalid certificate ID length, maximum is 25, this certificate will be skipped. Certificate id: {}	Please ensure that the params are correct and resubmit the request.
Invalid certificate ID, it contains illegal characters, this cert will be skipped. Certificate id: {}	Please ensure that the params are correct and resubmit the request.
WARNING: Nothing to update. The certification {} is already assigned to the employee with posId={}	Please ensure that the params are correct and resubmit the request.
ERROR: problem syncing cert with extref {} for emp with posId {} . Skipping this cert.	Please ensure that the params are correct and resubmit the request.

OperatingBudgetPlanService Available Methods

getLaborData

This method takes in a concept, store number and data type and returns the amount of the requested data type for each day by job code.

Call:

Name	Type	Required	Description/Example
Concept	Int	True	Concept ID - Provided by Fourth
storeNum	Int	True	Store ID

dataTypeID	Int	True	• 0 - LaborDollars • 1 - LaborPercent • 2 - SPLH • 3 - GPLH • 4 - LH100 • 5 - LaborHours
------------	-----	------	---

Response:

Name	Type	Description/Example
dataTypeID	Int	The budget type as an ID: • 0 - LaborDollars • 1 - LaborPercent • 2 - SPLH • 3 - GPLH • 4 - LH100 • 5 - LaborHours
days	Array	• dayOfWeek (Int) - Sunday (1) - Saturday (7) • Value (Float)
id	Int	The POS job ID

Example getLaborData response:

```

<dataTypeId>1</dataTypeId>
<jobs>
  <days>
    <dayOfWeek>1</dayOfWeek>
    <value>1.0</value>
  </days>
  <days>
    <dayOfWeek>2</dayOfWeek>
    <value>3.0</value>
  </days>
  <days>
    <dayOfWeek>3</dayOfWeek>
    <value>3.0</value>
  </days>
  <days>
    <dayOfWeek>4</dayOfWeek>
    <value>3.0</value>
  </days>

```

```

</days>
<days>
  <dayOfWeek>5</dayOfWeek>
  <value>3.0</value>
</days>
<days>
  <dayOfWeek>6</dayOfWeek>
  <value>3.0</value>
</days>
<days>
  <dayOfWeek>7</dayOfWeek>
  <value>1.0</value>
</days>
<id>16</id>
</jobs>

```

setLaborData

This method takes in a concept ID, store Num, and an array of budget objects to assign custom budgets to specific dates and budget types.

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
budget	Array	True	• dataTypeID (Int) ○ 0 - LaborDollars ○ 1 - LaborPercent ○ 2 - SPLH ○ 3 - GPLH ○ 4 - LH100 ○ 5 - LaborHours • Jobs (Array) ○ Days (Array) ■ dayOfWeek (Int) - Sunday (1) to Saturday (7) ■ Value (Int) ○ ID - Job POS ID

Example setLaborData call:

```
<concept>280</concept>
<storeNum>1100030</storeNum>
<budget>
  <dataTypeId>0</dataTypeId>
  <jobs>
    <days>
      <dayOfWeek>1</dayOfWeek>
      <value>5</value>
    </days>
    <id>1</id>
  </jobs>
</budget>
```

setLaborDataForWeek

This method takes in a concept ID, store Num, start date and end date in addition to an array of budget objects to assign custom budgets to specific dates and budget types.

Call:

Name	Type	Required	Description/Example
store_number	int	True	Store ID
concept	int	True	Concept ID - Provided by Fourth
startDate	Array	True	This is the start date of the work week. • Day (Int) • Month (Int) • Year (Int)
endDate	Array	True	This is the final date of the work week. • Day (Int) • Month (Int) • Year (Int)

budget	Array	True	• dataTypeID (Int) ○ 0 - LaborDollars ○ 1 - LaborPercent ○ 2 - SPLH ○ 3 - GPLH ○ 4 - LH100 ○ 5 - LaborHours • Jobs (Array) ○ Days (Array) ■ dayOfWeek (Int) - Sunday (1) to Saturday (7) ■ Value (Int) ○ ID - Job POS ID
--------	-------	------	---

Example setLaborDataforWeek call:

```

<concept>280</concept>
<storeNum>1100030</storeNum>
<startDate>
  <day>5</day>
  <month>1</month>
  <year>2020</year>
</startDate>
<endDate>
  <day>11</day>      <month>1</month>
  <year>2020</year>
</endDate>
<budget>
  <dataTypeId>0</dataTypeId>
  <!--Zero or more repetitions:-->
  <jobs>
    <!--Zero or more repetitions:-->
    <days>
      <dayOfWeek>4</dayOfWeek>
      <value>100</value>
    </days>
    <id>1</id>
  </jobs>
</budget>

```


HierarchyService Available Methods

Unsuccessful requests

Unsuccessful requests receive a response with HTTP status code 500 with a detailed error message in the body. Below is an example of a response that failed to be processed:

500 – if WSS-Password Type not set

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode xmlns:ns1="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
wssecurity-secext-1.0.xsd">ns1:InvalidSecurityToken</faultcode>
      <faultstring>Error in SecurityHeader: An invalid security token was
provided</faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

500 – general error

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring> Concept not found for concept external ref: {conceptId}</faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

The following errors can be returned in the faultstring element.

faultstring	Description
Concept not found for concept external ref: {conceptId}	The API was unable to locate information by concept. Please ensure that the params are correct and resubmit the request.

getConceptsV2

No additional errors

getStores

No additional errors

getGroups

No additional errors

getPunchRecordHistory

This new service returns the most recent punch records for a client and all their employees AND the history of edits for all punch records. The WSDL is available at:

<http://services.Fourth.com/api/services/PunchRecordService?wsdl>

Call:

Name	Type	Required	Description/Example
concept	int	True	Concept ID - Provided by Fourth
storeNum	int	True	Store ID
start	Array	True	- Day (Int) - Month (Int) - Year (Int)
end	Array	true	- Day (Int) - Month (Int) - Year (Int)

Response: data return for each employee

Name	Type	Description/Example
clientName	String	Store's Display Name
externalClientId	string	Store ID
empPOSid	int	Employee's POS ID
firstname	String	Employee's First Name
lastname	String	Employee's Last Name
hrId	long	Employee's HR ID

Response: data return for all punch records

Name	Type	Description/Example
adjuster	string	Full name of user who edited, deleted, or added a punch. Original punch records will not have an adjuster.
businessDate	string	Starting date for punch
cashTips	decimal	Total cash tips for punch
ccTips	decimal	Total credit card tips for punch
inTime	string	Start date and time of punch
jobcode	int	POS numeric Job Code ID
jobId	int	POS numeric identifier for the job

jobName	string	Job used for punch
outTime	string	End date and time of punch
punchTimeCardExternal ID	long	Unique ID linking punch records and timecards together. This ID is not unique across clients
punchType	string	Identifies a punch as: Original - Punch records that have not been edited/deleted/added Edited - Punch records that have been edited Deleted - Punch records that have been deleted Added - Punch records that have been manually added
svcCharge	decimal	Total SVC charge tips for punch
tippool	decimal	Total tip pool contribution for punch
totaltips	decimal	Total tips for punch ((cash tips + credit card tips + SVC charge tips) - Tip Pool)
typeDataTime	string	A date/time stamp to show when a punch was modified (ex: 2015-10-16T00:00:00-05:00)

Example getPunchRecordHistory response:

```

<clientName>Store Name </clientName>
  <empPOSId>111111</empPOSId>
  <externalClientId>111</externalClientId>
  <firstName>FirstName</firstName>
  <hrId>1111111</hrId>
  <lastName>LastName</lastName>
  <punchRecord>
    <adjuster/>
    <businessDate>2020-02-02</businessDate>
    <cashTips>0.00</cashTips>
    <ccTips>0.00</ccTips>
    <inTime>2020-02-02T14:19:09.000-06:00</inTime>
    <jobCode>12</jobCode>
    <jobId>1077602068</jobId>
    <jobName>Job12</jobName>
    <outTime>2020-02-02T20:33:22.000-06:00</outTime>
  <punchTimeCardExternalId>111111111111</punchTimeCardExternalId>
    <punchType>Original</punchType>
    <svcCharge>0.00</svcCharge>
    <tipPool>0.00</tipPool>
    <totalTips>0.00</totalTips>
    <typeDateTime>2020-02-02T20:33:23.000-06:00</typeDateTime>
  </punchRecord>
  <punchRecord>
    <adjuster>Adjuster Name</adjuster>

```



```
<businessDate>2020-02-06</businessDate>
<cashTips>0.00</cashTips>
<ccTips>0.00</ccTips>
<inTime>2020-02-06T10:30:00.000-06:00</inTime>
<jobCode>12</jobCode>
<jobId>1077602068</jobId>
<jobName>job12</jobName>
<outTime>2020-02-06T14:00:00.000-06:00</outTime>
<punchTimeCardExternalId>1111111111</punchTimeCardExternalId>
<punchType>Added</punchType>
<svcCharge>0.00</svcCharge>
<tipPool>0.00</tipPool>
<totalTips>0.00</totalTips>
<typeDateTime>2020-02-06T23:27:53.000-06:00</typeDateTime>
</punchRecord>
```